EZchain: A Secure Scalable Blockchain Protocol via Passive Sharding

EZchain Team

Abstract—Recently, many sharding blockchain protocols sacrifice some important attributes to improve scalability, which makes them complicated and insecure. Moreover, achieving constant communication costs in any network size is still a challenge for many sharding protocols.

Motivated by this, we present EZchain, a scalable blockchain protocol via "passive sharding" with proven validity and security. We redesign the Value-Centric Blockchains (VCB) framework to achieve the passive sharding that helps EZchain reach higher security than traditional sharding protocols. With fixed initialization parameters, the expected value of EZchain's communication cost reaches a constant level, and it is independent of network's size. Moreover, EZchain node's storage cost without beacon chains also approaches a constant and does not change with the increase of network's size and transactions. Cross-shard transactions, network sharding algorithm and anti-Sybil attack verifification are no longer needed in the passive sharding, thus EZchain is very concise and effificient. Our experiment uses a lightweight EZchain prototype and extends the experimental network size up to 100,000 nodes. The evaluation results show that EZchain satisfies all our analysis of its performance (the constant communication and non-beacon storage cost) in the large networks. In addition, the comparison experiment shows that EZchain has obvious advantages over the previous protocols in the long-term operation and large network environment.

Index Terms—Blockchain, distributed systems, distributed consensus, secure transactions.

1 Introduction

As a pioneer of distributed cryptocurrency, Bitcoin [1] is based on mining (i.e., Proof-of-Work, PoW) with the data structure called "blockchain" and attempts to construct a decentralized, unalterable global ledger. Although its security is adequately guaranteed [2], [3], [4], [5], Bitcoin is criticized for its poor scalability, centralization [6], [7], and huge energy waste problems [8]. Specifically, Bitcoin processes up to 7 transactions per second, and its utilization of the average available bandwidth is less than 0.2% [9]. Moreover, PoW mechanism leads to serious centralization of mining power [7]. In 2018, the total power consumption of Bitcoin accounted for 0.5% of global electricity consumption, which even exceeds that of some countries [8].

Maximizing system's efficiency is currently a very attractive challenge while balancing among decentralization, security and scalability. This inspired a lot of excellent works [10], [11], [12], [13], [14], [15], [16], [17], [18], which reflect different trade-offs: Separating the leader election from the consensus [11], which improves throughput to some extent but still has high transaction delay; Replacing the Nakamoto consensus with Byzantine Fault Tolerance (BFT) [19]—although it has a good transaction delay, the performance of the protocol is limited by BFT in the large network environment; In the leader election, replacing Proof-of-Stake (PoS) with verifiable random functions (VRF) [20] can solve the energy problem well [10], [13], [14], [15], yet it still requires linear Communication Cost Per Transaction (CCPT) in the number of nodes.

Sharding, another direction worthy of attention, has become the basis of many scalable solutions [21], [22], [23], [24], [25], [26]. It can divide the network, transactions, or states into several subsets (or shards), and the consensus algorithms run in various shards to reduce the need for bandwidth, storage and computing power, thus resulting in high efficiency and decentralization. But there are still many challenges: Excessive cross-shard transactions greatly affect system's efficiency; In shards, the security assumptions of the original consensus scheme no longer hold, thus most protocols need to strengthen the security model (e.g., OmniLedger [21] tolerates up to $n/4$ failures and guarantees that the BFT security model in the shards holds with a high probability: $1 - 10^{-6}$); Periodic verification of the nodes (e.g., PoW) is required by the protocol to prevent Sybil attacks [27] in the shards; Complex network sharding algorithms consume bandwidth and computing resources.

To address these challenges, in this paper, we propose EZchain, the first passive sharding blockchain protocol that genuinely satisfies constant CCPT. "Passive sharding" is a more generalized concept extracted from Value-Centric Blockchains (VCB) [22] (see Appendix H for details). As the sharding framework of EZchain, passive sharding does not require network sharding algorithms or even consensus in the shards, and the cross-shard transactions can be avoided.

As shown in Figure 1, EZchain is mainly composed of three sub-algorithms: Abstract Consensus (AC), Complete Consensus (CC), and Fast Consensus (FC). The AC algorithm, which is a plug-in in the EZchain system, can be any

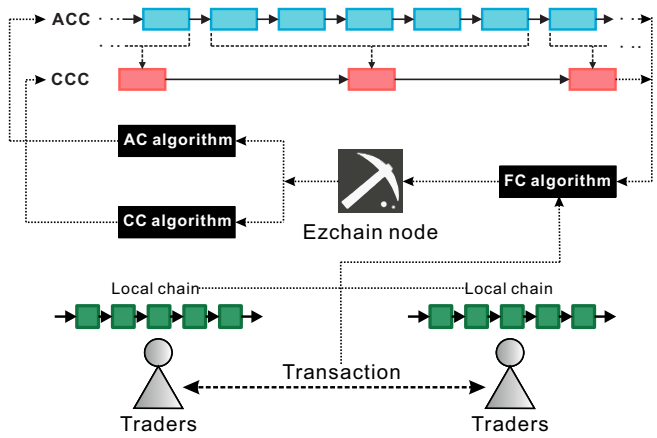

Fig. 1. Overview of EZchain system.

existing consensus algorithm that satisfies our requirements (e.g., Nakamoto consensus [1], Algorand [10], Byzcoin [12]), and it helps EZchain nodes reach a consensus on the Abstract Consensus Chain (ACC). The CC algorithm periodically reaches a consensus on the Complete Consensus Chain (CCC or checkpoints) in the system. As beacon chains, ACC and CCC provide a view of some necessary information for all global transactions. Therefore, when the traders perform a transaction, they can quickly verify the transaction's validity via the FC algorithm, which combines ACC, CCC and their local information to judge the transaction's legality, and then the passed transaction will be submitted to the EZchain node.

In terms of security, as a passive sharding blockchain protocol, EZchain's security and security model inherit from the AC algorithm. Unlike most sharding protocols, EZchain does not require any enhanced security model since the passive sharding requires the verification between transactors (i.e., FC) instead of the consensus in shards. And compared to VAPOR [22], EZchain is secure against denial-of-service (DoS) attacks.

With respect to validity, EZchain guarantees the asynchronous consistency and synchronous liveness of ACC and CCC with an exponentially small failure probability, which is inherited from the AC algorithm and maintained by the CC algorithms. And any illegal transaction (e.g., double-spending, invalid transaction) cannot pass the FC algorithm.

As to performance, we prove that EZchain has constant CCPT that is independent of the network's size. When using the optimal storage strategy, EZchain node only needs to prepare a constant non-beacon chain storage to cope with the increasing network's size and number of transactions, and this is a valuable property that is not achieved in other sharding protocols. In addition, EZchain also maintains the transaction latency and fairness of the AC algorithm. To further improve the performance, we present a series of optimization plug-in solutions as well.

Based on the CCPT evaluation strategy, we evaluate a lightweight EZchain with up to $100,000$ nodes to verify our theoretical analysis of EZchain's performance. The experimental results show that EZchain does satisfy constant CCPT. Except for some beacon information (i.e., ACC and CCC), the storage cost of EZchain node converges to a constant with increasing running time, and it does not increase with the expansion of network. EZchain can reach high throughput while keeping reasonable security. We also analyze the parameters that affect the scalability of EZchain. Finally, we compare EZchain with Vapor [22], Omniledger [21] and Algorand [10]. The results show that the communication and storage cost of EZchain is independent of network size and system running time, while other protocols cannot achieve this.

2 Background

2.1 Various Consensus' Communication Costs

In terms of the measurement of communication costs, we use the *communication cost per transaction* (CCPT) proposed by Ren et al. in [28]. CCPT is not explained in detail in [28], and thus, we give a formal definition of it in Definition 1.

Definition 1 (CCPT). A communication cost per transaction (CCPT) mainly includes:

- **CCPT of transaction**: For any transaction txn, denote T_{txn} as the timestamp when txn first appeared in the system (i.e., the time when txn was generated), and T_{end} as the end time of the system (theoretically, T_{end} approaches $+\infty$). Then in the time span from T_{txn} to T_{end}, the time that all nodes in the system use bandwidth resources to send txn is the CCPT value of txn, which is also denoted as $CCPT_{txn}$.
- **CCPT of protocol**: For any protocol X (e.g., $X =$ Bitcoin, VAPOR or EZchain), denote T^X_{begin} as the time when the protocol starts to run, and T^X_{end} as the end time of X (theoretically, T^X_{end} approaches $+\infty$). And let $\mathcal{X}_{txn}$ be the set of all transactions included in X in the time span from T^X_{begin} to T^X_{end}. Then the CCPT of protocol X (denoted as $CCPT_X$) is

$$\lim_{T^X_{end}\to+\infty} \frac{\sum_{txn\in\mathcal{X}_{txn}} CCPT_{txn}}{|\mathcal{X}_{txn}|},$$

  where $|\mathcal{X}_{txn}|$ is the number of the elements in $\mathcal{X}_{txn}$.

Nakamoto consensus-based protocols generally require $O(N)$ CCPT, because they require each transaction to be recorded by all nodes. In order to reach a consensus in the asynchronous system, the traditional Practical Byzantine Fault Tolerance (PBFT) protocol [19] requires $O(N^2)$ CCPT, and the Hotstuff protocol [29] uses the threshold signature and the four-step consensus to completely reduce the PBFT's CCPT to $O(N)$. On this basis, BFT-based blockchain protocols generally also have $O(N)$ CCPT, since they also require all nodes to record the global ledger.

In order to further reduce CCPT and improve scalability, two types of solutions have been proposed— layer-2 and sharding. The former deviates from the framework of the consensus agreement, moves the transactions to the off-chain (i.e., layer-2), and treats the blockchain (layer-1) as a trusted third party to ensure the consistency of the final result. Although layer-2 reduces communication cost to $O(1)$, the validity of transactions requires additional mechanisms (e.g., additional verification nodes [30], [31], deposit [32], [33]) to guarantee. As mentioned earlier, the principle of sharding reduces its CCPT to $o(N)$, yet many

sharding protocols require stronger security assumptions to prevent malicious attacks for the single sharding, and in some extreme cases (e.g., a large number of cross-shard transactions) leads to a drop in efficiency.

Although the spontaneous sharding [28] proposed by Ren et al. also achieves $o(N)$ CCPT, its communication consumption still increases with the expansion of the network size (the traditional sharding protocols can increase the number of shards to keep the CCPT not increasing with the increase of N, yet this may cause other burdens). The need for storage of spontaneous sharding is also increasing as transactions progress. And the spontaneous sharding's core idea—VCB will also be introduced in the next subsection.

2.2 Value-Centric Blockchain

Ren et al. propose the concept of "Value-Centric Blockchain (VCB)" [22], which has many advantages compared with the traditional sharding. Our protocol is also based on the improved VCB.

The data structure of VCB is similar to the "off-chain" protocol. The entire network only needs to reach the consensus on a abstract chain, and the node's transactions corresponding to the abstract chain are stored locally. The value (or coin) with its ownership is the core of the VCB. When trading, VCB's proof and verification algorithms are used to ensure the legitimacy of the transaction by verifying the value's ownership. Below we briefly introduce these elements of VCB and its comparison with EZchain:

Abstract chain: In VCB, the *abstract chain*, which is confirmed by the AC algorithm, is composed of transactions' abstracts, and it orderly records the abstracts of all submitted transactions. The abstract chain is denoted by $\mathcal{B} = \{B_1, B_2, ...\}$, where B_i is the i-th abstract block. Note that B_i is used to represent the clock state of the entire system as well (e.g., at B_i).

Value and ownership: There are N nodes in the whole network, which are denoted by $1, 2, ..., N$. Then at the state B_i, for the value $v_j, j = 1, 2, ...$, its owner is $O(v_j, B_i) \in 1, 2, ..., N, NA$, where NA means no owner. The initial value distribution $\{[v_j, O(v_j, B_1)] : \forall v_j\}$ is recorded in the genesis block B_1. In addition, VCB contains two algorithms: the verification algorithm $\texttt{GetOwner}(v_j, B_i, p)$ and the proof algorithm $\texttt{Proof}(v_j, B_i)$. If $p = \texttt{Proof}(v_j, B_i)$, then $\texttt{GetOwner}(v_j, B_i, p)$ returns $O(v_j, B_i)$, otherwise it returns "Fail". In summary, **Proof** provides a legal proof that the node $O(v_j, B_i)$ owns v_j at B_i. **GetOwner** is used to verify the proof p and return the owner of v_j at B_i.

Transaction (or value's transfer): A legal transaction is denoted by $tx(v_j, x \rightarrow y)$, then at a certain state B_i, $O(v_j, B_i) = x$ and $O(v_j, B_{i+1}) = y$ after this transaction. Specifically, if x holds v_j at $B_t, (t < i + 1)$, then x must have p such that $\texttt{GetOwner}(v_j, B_t, p) = x$. If x wants to transfer v_j to y at B_i, then x should send y: p makes $\texttt{GetOwner}(v_j, B_i, p) = x$, i.e., $p = \texttt{Proof}(v_j, B_i)$; and p' makes $\texttt{GetOwner}(v_j, B_{i+1}, p') = y$. y needs to detected: $\texttt{GetOwner}(v_j, B_i, p) = x$ and $\texttt{GetOwner}(v_j, B_{i+1}, p') = y$.

Comparison with Ezchain: The idea of passive sharding, which allows the transactions' path to adaptively divide the set of nodes and optimize the consensus process, comes from VCB. However, VCB and its first instance, VAPOR, did not achieve their design goal, i.e., a constant CCPT (in fact, they only satisfy $o(N)$, see Appendix F for details). Based on newly added checkpoint consensus (i.e., CC), redesigned the verification (i.e., FC), and passive sharding that inherits from the design ideas of VCB, EZchain can achieve constant level CCPT and storage consumption. EZchain also compensates for some VCB's security vulnerabilities (e.g., vulnerable to DDoS attacks). In addition, we also provide a variety of performance optimization plug-ins to help further improve the performance of EZchain.

And finally, we make some additions to the definition of a valid VCB in [22]:

Definition 2 (Valid VCB). We give three properties:

- **Ownership**: If x has a value v_j in the time from B_{i_1} to B_{i_2}, then x must has $p = \texttt{Proof}(v_j, B_i)$, $i_1 \leq i \leq i_2$, such that $\texttt{GetOwner}(v_j, B_i, p) = x$. And any node can execute the **GetOwner** after receiving p.
- **Liquidity**: The value v_j can only be transferred by the current owner x through the legal transaction $tx(v_j, x \rightarrow y)$ in bounded time, i.e., if $O(v_j, B_i) = x$, then only x can make $O(v_j, B_k) = y$, $k > i$, and $k - i$ is bounded by a constant.
- **Authenticity**: At the same time, any value can only be owned by one legal owner at most, i.e., for $\forall v_j, B_i$, $O(v_j, B_i) \in \{1, 2, ..., N, NA\}$.

Then a VCB protocol is valid if and only if Ownership and Authenticity are guaranteed under asynchronous network settings, and Liquidity is guaranteed under synchronous network settings.

3 Model and Goal

3.1 Model

System Model We consider a peer-to-peer network with N nodes that need to establish identities through some Sybil-attack-resistant mechanisms [34]. The system has standard cryptographic assumptions (e.g., public key signatures, collective signatures). The running time of EZchain is divided into multiple epochs, each of which contains several rounds (i.e., the AC algorithm is run in each round, and the CC algorithm is run at the end of each epoch). The length of the epoch is controllable. Depending on the purposes (e.g., higher average throughput, more bandwidth resources, higher security), the epoch can be adjusted to any reasonable length.

Network Model Referring to the model in previous works [10], [13], [21], [23], we assume that the communication and broadcast (via gossiping) between honest nodes are well connected and synchronized. Specifically, for broadcast message between honest nodes, it will be sent to all honest nodes within a fixed time Δ, and we regard Δ is in the scale of minutes, so Δ is used for delay-insensitive operations (i.e., CC algorithm). In other parts (e.g., FC algorithm), we assume that the time-outs of the synchronization channel between nodes increase exponentially [21] to achieve low latency and responsiveness to the greatest extent.

Threat Model EZchain tolerates up to $f = \lambda N$ Byzantine nodes, where λ is determined by the AC algorithm, i.e., EZchain has the same tolerance bounds as the AC algorithm. The Byzantine node can make any deviation from

the protocol. We also assume that the adversary is slowly adaptive (similar to [12], [21], [25], [35]), namely, the ability to select Byzantine nodes has a high delay (e.g., the set of Byzantine nodes in the current epoch cannot be changed). In addition, the adversary cannot break our cryptographic assumptions in polynomial-time.

Value and Transaction Model Each node is assigned the same amount of legal unit currency (value) when it is initialized. The generation of the transaction is random, i.e., each node generates the transaction with the same probability per unit time. To simplify the model, we let each transaction consume one unit of currency (value). And the other party (i.e., receiver) of each transaction is randomly selected as well.

Rationality of Value Owner Model The value owner is rational when dealing with his value, specifically: 1) When the owner legally holds the value, as long as he wants, he can provide corresponding evidence to prove that he is the legal owner of this value. 2) When the owner wants to transfer this value to another node X, then he has the responsibility to provide X with corresponding evidence to prove that he legally holds this value, and X has the responsibility to verify the legality of this evidence.[1]

3.2 Goal

Our protocol needs to achieve the following goals at the same time:

Validity: EZchain should satisfy Definition 2 (In fact, EZchain does not satisfy Definition 2 only with an exponentially small probability).

Efficiency: EZchain's CCPT should be equal to $O(\omega)$, where ω is independent of N. If the parameters of our model are fixed, then ω is almost stable as a constant; EZchain node's maximum storage required for the non-beacon information (i.e., in addition to ACC and CCC) is independent of N and transactions, and it will also eventually stabilize near a constant.

Security: EZchain should have the same security level as that of the AC algorithm, and it should not introduce additional security assumptions.

Transaction Delay: EZchain should have almost the same average transaction delay as that of AC algorithm.

4 Protocol Design

4.1 Overview

As mentioned before, our protocol includes three sub-algorithms: Abstract Consensus (AC) algorithm, Complete Consensus (CC) algorithm and Fast Consensus (FC) algorithm. Below we introduce the three sub-algorithms and the main algorithm.

AC Algorithm (Subsection 4.2) The consensus part of the specific AC algorithm, which is based on existing consensus protocol, is not the focus of this work. We mainly use the AC to make the whole network reach a consensus on the beacon chain—Abstract Consensus Chain (ACC), and

1. The rationality of value owner model is not stronger than the security assumptions of traditional blockchains. In fact, it has even been implicitly used by many blockchain systems. For more information, please refer to [22].

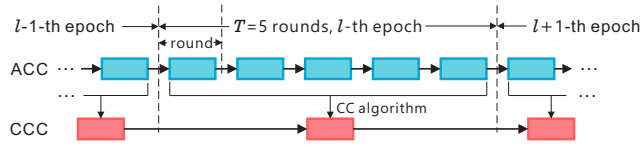

Fig. 2. The blue chain represents ACC and the red chain represents CCC.

the ACC records the abstract content of all transactions (including illegal transactions). The AC algorithm is executed in EZchain's minimum period unit (i.e., *round*).

CC Algorithm (Subsection 4.3) The CC algorithm is periodically executed after each *epoch*, and the epoch's period is T. Its aim is to conduct a comprehensive verification of all transactions in this epoch's rounds and produce a checkpoint of Complete Consensus Chain (CCC or checkpoints). The relationship between CCC and ACC is shown in Figure 2. The checkpoint records the indices of all illegal transactions in this epoch. In other words, the transactions that are not included in the checkpoint are legal. In the process of value transfer (transaction), the trader can quickly verify the legitimacy of the value according to the latest checkpoint instead of pursuing its origin.

FC Algorithm (Subsection 4.4) Similar to the idea of lightning network, the FC algorithm allows transactors to complete transactions quickly with the aid of beacon chains (ACC and CCC). For the information that is required by verification and not covered by the beacon chains, the transaction initiator is required to provide sufficient proof (according to local information) to ensure the legitimacy of the value. Unlike the lightning network, the FC algorithm runs on Layer-1, and the legitimacy of all transactions is guaranteed by the consensus algorithm.

Main Algorithm The main algorithm of EZchain is shown in Algorithm 1, here we denote B_k^l as the k-th abstract block of the l-th epoch (i.e., $B_k^l = B_{k+\sum_{i=1}^{l-1}\#(\mathcal{B}_i)}$, where$\#(\mathcal{B}_i)$ is the number of abstract blocks in the i-th epoch).

4.2 Abstract Consensus (AC) algorithm

The AC algorithm makes all nodes reach consensuses on the abstracts of all collected transactions in each round. An existing consensus scheme $\mathcal{S}_A$ needs to be selected by the AC algorithm as its carrier (e.g., Nakamoto Consensus, BFT Consensus). Below we introduce the contents of the AC algorithm.

Choice of $\mathcal{S}_A$ $\mathcal{S}_A$ needs to satisfy the following conditions, where $\mathcal{B} = \{B_1, B_2, ...\}$ denotes the ACC, and B_i is the i-th abstract block:

- *Asynchronous Consistency*: In the case of arbitrary network's delay, different honest nodes cannot have different viewpoints on $\mathcal{B}$;
- *Synchronous Liveness*: If the network's delay is bounded by a constant, the legal transactions' abstracts will eventually be confirmed by $\mathcal{B}$.
- *immutability*: The history content of ACC cannot be rewritten by any node.
- *compatibility*: The validity and security of $\mathcal{S}_A$ do not depend on its original chain's data, i.e., $\mathcal{S}_A$ is still

```
new Process:
for true do
    if node i needs to transfer value then
        Run FC algorithm; \\ The FC algorithm helps traders quickly verify and complete transactions.
    end
end
new Process end

k ← 0, l ← 1; \\ Parameter initialization.
for true do
    if k + 1 < T then
        Run AC algorithm; \\ A consensus is reached on the ACC's k + 1-th block in l epoch via the AC algorithm.
        k ← k + 1;
    end
    else
        if i ∈ temporary committee then
            Run CC algorithm; \\ CC algorithm completes the checkpoint consensus of l epoch, which includes all illegal transactions in this epoch.
        end
        k ← 0, l ← l + 1;
    end
end
```

Algorithm 1: Main algorithm for $\forall$ node i at B_k^l

```
function Committee (epoch l):
B_l ← {all the abstract blocks in the l-th epoch};
TC_l ← {L_i | ∀ i s.t. B_i ∈ B_l}, where L_i is the leader of i-th round;
return TC_l;
```

```
function Leader (temporary committee TC_l, previous leader L_i):
if L_{i+1} ∉ TC_l ∧ L_i = L_0^l then
    L_1^l ← the first leader in TC_l; \\ L_i^l represents the i-th leader in the l-th epoch.
    return L_1^l;
end
else
    return L_{i+1};
end
```

Algorithm 2: Utilities (for CC algorithm in l-th epoch)

correct and secure after modifying the content of its original chain as ACC.

- *Fairness*: The leader election in $\mathcal{S}_A$ is "fair", i.e., the probability that the leader is failed cannot exceed the tolerance boundary of $\mathcal{S}_A$'s security model (e.g., 1/3 of the BFT Consensus).

In this paper, we do not explicitly specify $\mathcal{S}_A$, for any scheme that satisfies the above five conditions can be employed.

Abstract Consensus Blocks In EZchain, node x packages its own information set $inf = (txs, prf)$ in each round, where txs is the x's unconfirmed transactions set and prf is the needed txs's proofs set (the data structure of prf is the same as that of txs, and the content of prf will be described in detail in Subsection 4.4 and Appendix G). x submits inf with the abstract $A(x)$ to the current leader, and waits for the leader's confirmation (note that only the legal abstract needs to be added to the ACC, and inf does not need to be consensus). Specifically, at B_i, for the x's unconfirmed transactions $txn_1 = tx_1(v_j, x \to y), txn_2 = tx_2(v_k, x \to z), \cdots$, x temporarily stores $txs = \{[v_j, y, sn_1], [v_k, z, sn_2], \cdots\}$, where sn is x's internal serial number of this transaction, and $prf = \{p_1 = \texttt{Proof}(v_j, B_{i+1}) \cup p_2 = \texttt{Proof}(v_k, B_{i+1}) \cup \cdots\}$ if it is needed. Then x can periodically submit inf with the abstract $A(x)$,

$$A(x) = [x, H(pk_x), Sig_x(H(pk_x) \mid MR(txs))],$$

to the abstract block B_{i+1} in each round, where $H(pk_x)$ is the hash of x's public key, $MR(txs)$ is the Merkle root of txs encrypted by x's private key, and Sig_x is the x's digital signature. Note that at most one $A(x)$ is included in each B_{i+1}, and multiple different $A(x)$s in the same B_{i+1} are considered illegal.

Personal Transaction Chain If $A(x)$ is included in a block $B_i \in \mathcal{B}$, then we denote it by $A_i(x)$, and x's information set inf corresponding to $A_i(x)$ is denoted by $inf_i(x)$. The $txs_i(x)$ and $prf_i(x)$ have the similar definitions to $inf_i(x)$. Since $\mathcal{B}$ is immutable, then x also has a personal transaction chain $\mathcal{PB}_x = \{txs_i(x) \mid \forall i$, and if x does not submit an abstract to B_i, then $txs_i(x) = \varnothing\}$.

Pre-Legality Test As mentioned earlier, $inf_i(x)$ is submitted along with $A(x)$. The leader is responsible for detecting the legality of $inf_i(x)$: $inf_i(x)$'s abstract is $A(x)$; no illegal transaction in $txs_i(x)$; and prf is legal (i.e., pass the test of `GetOwner`). The leader ignores illegal $inf_i(x)$'s $A(x)$. And the $inf_i(x)$ is temporarily stored by the leader to prepare for the CC algorithm (see Section 4.3 for details).

4.3 Complete Consensus (CC) algorithm

Firstly, we denote CCC by $\mathcal{CB} = \{CB_1, CB_2, ...\}$, and the l-th epoch's temporary committee by TC_l (see Algorithm 2). The CC algorithm mainly makes the whole network reach a consensus on an "*illegal transactions list of the l-th epoch's ACC*" (i.e., CB_l). For detailed content, see Algorithm 3 (Part. A) and Algorithm 4 (Part. B), respectively.

Ideally, for $\forall\ L_i \in TC_l$ will broadcast their own inf_i, and there is no need to collect proofs from other nodes (because the transaction submitted to ACC by L_i must be verified by L_i as well, L_i must have its legal proof). However, there are still malicious and offline nodes, resulting in additional bandwidth consumption. For this situation, we can set a reasonable incentive strategy to avoid this part of the waste (yet this is beyond the scope of this work, we do not discuss it).

```
Initialize set CB_l = ∅, temporary committee
  TC_l ← Committee (l) and TC_l's leader
  L_TC = L_0^l; \\ L_0^l is only used for initialization
  and does not represent a specific leader.
if the latest CB_j satisfies j < l − 1 then
    Waiting for the completion of the l − 1-th
      epoch's CC algorithm;
end
for ∀ node t ∈ TC_l do
    t broadcasts its collected inf_i (∀ i s.t. B_i ∈
      B_l)= {inf_i(x)|∀x, s.t. inf_i(x) ≠ ∅} with
      t's signature in TC_l (inf_i is collected by t in
      AC algorithm, see Section 4.2);
    Wait for a fixed time τ_1 (τ_1 is related to the
      synchrony assumption);
    for ∀ i and ∀x s.t. B_i ∈ B_l ∧ inf_i(x) ≠ ∅ do
        if inf_i(x) is not received by t after time τ_1
          according to A_i(x). then
            t add the indices of all transactions,
              which ∈ inf_i(x)'s txs_i(x), to CB_l;
        end
        else
            for ∀ txn_k = tx(v_j, x → y) ∈ txs_i(x)
              and txn_k's proof p_k ∈ prf_i(x) do
                if GetOwner(v_j, B_{i−1}, p_k −
                  {txs_i(x)}) ≠ x ∨
                  GetOwner(v_j, B_i, p_k) ≠ y then
                    Add txn_k's index to CB_l;
                end
            end
        end
    end
end
L_TC ← Leader (TC_l, L_TC);
Leader L_TC broadcasts its CB_l with its
  signature in the TC_l to obtain the collective
  signature of all the TC_l's members who agree
  with the CB_l (i.e., L_TC's CB_l is the same as its
  CB_l);
If the operation of L_TC is not completed within
  a fixed time τ_2, then L_TC ← Leader (TC_l,
  L_TC);
L_TC broadcasts the CB_l with the valid
  collective signature, i.e., the CB_l is signed by
  more than 1/2 nodes of the TC_l, to the entire
  network;
```

Algorithm 3: CC algorithm in epoch l (Part. A)

```
for ∀ node e in the network do
    for ∀ txn = tx(v_j, A → B), whose index
      ∈ CB_l do
        Denote B_k as that txn happens at B_k;
        if e = A ∨ e = B then
            e broadcasts txn and its proofs:
              p_1 =Proof(v_j, B_{k−1}) and
              p_2 =Proof(v_j, B_k) with e's
              signature to the TC_l;
        end
    end
end
Wait for a fixed time τ_3;
for ∀ node t ∈ TC_l do
    for ∀ txn = tx(v_j, A → B), whose index
      ∈ CB_l do
        if ∃ node e s.t. (e = A ∨ e = B) ∧ e
          submits txn with p_1 and p_2 to TC_l within
          the time τ_3 then
            if GetOwner(v_j, B_{k−1}, p_1) = A ∧
              GetOwner(v_j, B_k, p_2) = B ∧ e's
              signature is legal then
                t remove the index of txn from
                  CB_l;
            end
        end
    end
end
L_TC broadcasts its new CB'_l with its signature
  in the TC_l to obtain the collective signature of
  all the TC_l's members who agree with the
  CB'_l;
If the operation of L_TC is not completed within
  a fixed time τ_4, then L_TC ← Leader (TC_l,
  L_TC);
L_TC broadcasts the CB'_l with the valid
  collective signature to the entire network;
for ∀ node e in the network do
    if the received CB'_l is signed by more than 1/2
      nodes of the TC_l then
        e adds the CB'_l to its CB;
    end
end
```

Algorithm 4: CC algorithm in epoch l (Part. B)

In addition, the CC algorithm may cause the congestion of messages' transmission and scarcity of bandwidth resources, because in the TC_l, the inf information of one epoch are intensively broadcast. But this is easy to solve. We can redesign the broadcast and collection phase of inf in TC_l: leader L immediately broadcasts its collected inf to the previous leader of this epoch when L takes office (i.e., the round during L is selected as the leader), and also sends its inf to the future leaders of this epoch when they take office. Although this may require some additional bandwidth consumption, it effectively solves the problem of aggregated broadcast.

Finally, for the repeated transmission problem in the CC algorithm (i.e., there is a large amount of redundancy in the inf messages broadcasted by each node in TC_l, e.g., for two legal transactions $tx(v, A \to B)$ at B_k, whose proof is p_{AB} and $tx(v, B \to C)$ at B_{k+1}, whose proof P_{BC}, then $P_{AB} \subset P_{BC}$). We give a reasonable solution in Appendix G, which makes each transaction (whether as the txn or the part of prf) be broadcasted at most once in each TC_l.

O_i sends $p =$ **Proof**(v_j, B_k) to O_{i+1};
if **GetOwner**$(v_j, B_k, p) \neq O_i$ **then**
  O_{i+1} reject the transaction, **Return** Fail;
end
O_i sends $txs_{k+1}(O_i)$ to O_{i+1};
if
  GetOwner$(v_j, B_{k+1}, p \cup \{txs_{k+1}(O_i)\}) \neq O_{i+1}$
  then
  O_{i+1} reject the transaction, **Return** Fail;
end
Return Success;

Algorithm 5: FC algorithm for O_i sending v_j to O_{i+1} at B_{k+1}

4.4 Fast Consensus (FC) algorithm

The FC algorithm is a fast value transfer scheme between transactors. For a transaction $tx(v_j, O_i \rightarrow O_{i+1})$, the FC algorithm only needs the value owner O_i to transmit the necessary information to O_{i+1} to complete this transaction with the help of ACC and CCC. Specifically, if O_i wants to transfer v_j to O_{i+1} at B_{k+1}, then O_i needs to transmit proofs $p_1 =$ **Proof**(v_j, B_k) and $p_2 = p_1 \cup txs_{k+1}(O_i)$, $s.t.$ **GetOwner**$(v_j, B_{k+1}, p_2) = O_{i+1}$, to O_{i+1}. And O_{i+1} can use **GetOwner** to check the legality of the transaction. The specific content of the FC algorithm is shown in Algorithm 5.

In fact, FC algorithm only needs to perform the second verification (i.e., p_2), but the two-step verification in Algorithm 5 (i.e., p_1, p_2) is more beneficial to improve the protocol's efficiency and reduce the transaction delay (see the analysis of transaction delay in Subsection 5.2.2 for details).

Next we focus on these two algorithms—**Proof** and **GetOwner** in FC algorithm. **Proof**(v_j, B_k) outputs the valid proof p about the v_j's owner at B_k or "Fail" (i.e., only the legal owner of v_j can provide legal p); **GetOwner**(v_j, B_k, p) is used to check the validity of p, and it returns $O(v_j, B_k)$ if and only if $p =$ **Proof**$(v_j, B_k) \wedge p \neq$ "Fail", otherwise returns "Fail". Note that EZchain's **Proof** combines the checkpoints (CCC) and outputs the shortest proof p, and **GetOwner** also needs to combine the latest checkpoint to make judgments. However, the original VCB's **Proof** outputs the redundant p [22], which is one of the reasons that limit its performance.

The specific **Proof** algorithm is shown in Algorithm 6. The **Proof** first finds the transfer record of v_j corresponding to the latest checkpoint, and then judges whether it can provide legal proof based on these records. And Figure 3 visually shows two cases in Algorithm 6: $L \neq \varnothing$ and $L = \varnothing$. For convenience, we denote the proof in Case 1 as "*checkpoint proof*" and the proof in Case 2 as "*complete proof*" (in fact, all proofs without checkpoint can be called "complete proof").

The specific content of **GetOwner** is shown in Algorithm 7. **GetOwner** first performs some basic tests on the proof p and refers to $\mathcal{B}$ to check p's integrity and correctness. Secondly, check the v_j's transfer process in p for errors and double-spending attacks. Finally, if it passes the tests, then output O_i as the owner of v_j at B_k.

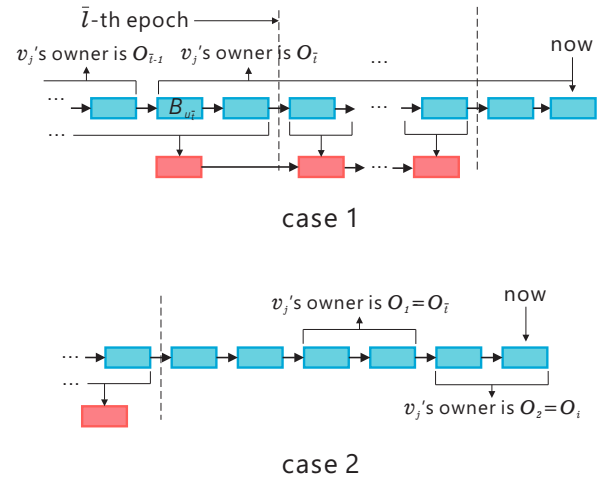

Fig. 3. This figure shows two cases of Algorithm 6. Case 1: The transfer record of v_j contains at least one checkpoint, which can help the transactors to provide more concise proof, i.e., they only need to provide the evidence of "they do not transfer v_j after the checkpoint $CB_{\bar{l}}$"; Case 2: This is the case when v_j was "born". v_j's transfer record does not contain any checkpoint, so all the v_j's transfer records need to be provided. Case 1 is generally more common than Case 2.

5 Protocol Analysis

This section analyzes EZchain in terms of validity, security and performance.

5.1 Validity and Security

In order to prove that EZchain is a valid VCB protocol, we first prove the validity of CC algorithm and FC algorithm.

Theorem 1 (CC algorithm's validity). CC algorithm can make all nodes of the network reach a consensus on CCC with an exponentially small failure probability, and satisfies:

- Asynchronous consistency: For arbitrary network delay, any two different honest nodes cannot have different viewpoints on $\mathcal{CB}$.
- Synchronous liveness: For bounded network delays, the submitted legal CB_i will eventually be included by $\mathcal{CB}$.
- Correctness: For any transaction $txn = tx(v_j, x \rightarrow y)$, whose abstract is included by ACC, then: (i) If at least one of x and y is honest, then txn's index $\notin \mathcal{CB}$; (ii) If txn is illegal (i.e., no legal proof), then txn's index $\in \mathcal{CB}$.

Note that the validity of CC algorithm implicitly depends on the assumption of *synchrony*, yet it is included by the AC algorithm's security assumptions. And the proof of Theorem 1 is shown in Appendix A.1.

Next, we analyze the validity of the FC algorithm. And the proof of Theorem 2 is given in Appendix A.2.

Theorem 2 (FC algorithm's validity). The legal owner O_i of the value v_j always has p $s.t.$ **GetOwner**$(v_j, B_k, p) = O_i$, where B_k is the period in which O_i owns v_j. Only O_i can independently transfer v_j to any next owner O_{i+1} through **Proof**,

.

Suppose that $T = \{B_{u_1}, B_{u_2}, \cdots, B_{u_{i+1}}\}$, where B_{u_t} means v_j's t-th owner get v_j at B_{u_t}, thus $B_k = B_{u_{i+1}-1}$, i.e., $T = \{B_{u_t} \mid$ transaction $tx(v_j, O_{t-1} \to O_t), i \geq t > 1$, happens at $B_{u_t}\} \cup \{B_{u_1} \mid B_{u_1}$ is the initial block of $v_j\} \cup \{B_{u_{i+1}} \mid O_i$ will transfer v_j to O_{i+1} at $B_{u_{i+1}}\}$. And let $L = \{l \mid \forall\, l\ s.t.\ (\exists\, t\ s.t.\ B_{u_t} \in \mathcal{B}_l, i \geq t \geq 1)\}$, where $\mathcal{B}_l = \{\forall\, B_i$ in l-th epoch, and note that if the l-th epoch does not end, then $\mathcal{B}_l = \varnothing\}$;
if $L \neq \varnothing$ **then**
  Let $\bar{l}$ denote the largest element of the set L and $\bar{t}$ denote the largest t such that $B_{u_t} \in \mathcal{B}_{\bar{l}}$;
end
else if $L = \varnothing$ **then**
  $\bar{t} \leftarrow 1$;
end
$index \leftarrow \bar{t}$;
for $index \leq i$ **do**
  $w \leftarrow u_{index},\ count \leftarrow 0$;
  if $txs_w(O_{index-1})$*'s abstract does not match* $\mathcal{B}$ $\vee$ *it has double-spending (i.e.,* v_j *is not only transferred to* O_{index} *by* $O_{index-1}$ *in* $txs_w(O_{index-1})$*)* **then**
    Return Fail;
  end
  for $w \leq u_{index+1}$ **do**
    if *the abstract of* $txs_w(O_{index})$ *is included in* B_w **then**
      if $txs_w(O_{index})$*'s abstract does not match* $B_w \vee txs_w(O_{index})$ *exists in* $\mathcal{CB}$ **then**
        Return Fail;
      end
      $count \leftarrow count +$ number of transfers of v_j in $txs_w(O_{index})$;
    end
    $w \leftarrow w + 1$;
  end
  if $count \neq 1$ **then**
    Return Fail;
  end
  $index \leftarrow index + 1$;
end
Return $Proof = \{txs_{u_{\bar{t}}}(O_{\bar{t}-1})\} \cup (\bigcup_{v=\bar{t}}^{i} \{txs_w(O_v) \mid u_v \leq w \leq u_{v+1}\})$;

Algorithm 6: Proof algorithm **`Proof`**(v_j, B_k) for O_i sending v_j to O_{i+1} at B_{k+1}

and O_{i+1} can detect the legality of this transaction through **`GetOwner`**.

Finally, we combine the above two theorems to give the validity conclusion of EZchain, i.e., EZchain is a valid VCB protocol (Definition 2). And the proof of Theorem 3 is given in Appendix A.3.

Theorem 3 (EZchain's validity). EZchain satisfies:

- The Ownership and Authenticity of Definition 2 hold;
- Under the assumption of synchrony, the Liquidity of Definition 2 holds.

with an exponentially small failure probability.

if *the data structure of* p *is not* $txs_i(x)\ \vee\ \exists\, txs_i(x) \in p\ s.t.\ txs_i(x)$ *does not pass the basic tests (key, signature, abstract, etc.)* **then**
  Return Fail;
end
Then we have $p = \{txs_{u_v}(O_{v-1}), txs_{u_v}(O_v), txs_{u_v+1}(O_v), \cdots, txs_{u_{v+1}}(O_v), txs_{u_{v+1}}(O_{v+1}), \cdots, txs_{k-1}(O_i), txs_k(O_i)\}$;
if p *is a checkpoint proof* **then**
  if *there is no checkpoint between* B_{u_v} *and* $B_{u_{v+1}}$ **then**
    Return Fail;
  end
  else if $\exists\, txs_i(x) \in p\ s.t.\ txs_i(x)$ *included by* $\mathcal{CB}$ **then**
    Return Fail
  end
end
$index_1 \leftarrow v$;
for $index_1 \leq i$ **do**
  $index_2 \leftarrow u_{index_1},\ count \leftarrow 0$
  for $index_2 \leq u_{index_1+1}$ **do**
    $count \leftarrow count +$ the number of transfers of v_j in $txs_{index_2} O(index_1)$;
    if $index_2 = u_{index_1+1} \wedge count \neq 1$ **then**
      Return Fail;
    end
    if $index_2 = u_{index_1+1} \wedge tx(v_j, O_{index_1} \to O_{index_1+1}) \notin txs_{index_2}(O_{index_1})$ **then**
      Return Fail;
    end
    if $index_2 \neq u_{index_1+1} \wedge count > 0$ **then**
      Return Fail;
    end
    $index_2 \leftarrow index_2 + 1$;
  end
  $index_1 \leftarrow index_1 + 1$;
end
Return $owner = O_i$;

Algorithm 7: Verification algorithm **`GetOwner`**(v_j, B_k, p) for O_i sending v_j to O_{i+1} at B_{k+1}

In terms of security, the CC and FC algorithms of our protocol do not contain additional security assumptions. And according to Theorem 3, our protocol is correct and valid. Therefore, EZchain has the same security and security assumptions as the AC algorithm (here we believe that the probability of CC algorithm's failure is negligible).

In addition, EZchain is less susceptible to DoS attacks than VAPOR (see Appendix E for details).

5.2 Performance

We here focus on several aspects of our protocol: throughput, decentralization (storage), fairness, and transaction's confirmation latency.

5.2.1 CCPT and Scalability

Considering CCPT from the perspective of single transaction will make analysis easier—i.e., CCPT is equivalent to the average total number of one transaction's transmissions in the whole network. In addition, this definition of CCPT helps us achieve more generalized and lighter-weight experiments (see Section 6). Below we give the result of EZchain's CCPT:

Theorem 4. The CCPT of a transaction txn in EZchain is $O(m) + O(k) + O(h) + O(m \cdot f)$, where m is the size of the temporary committee, k is the average number of times of the value's transfers in one epoch, h is the average number of values owned by a node when txn is confirmed, f indicates how many future temporary committees that need to reach consensus on txn (txn is transmitted as part of the prf in the future).

Proof *Figure 4 shows a more intuitive explanation of $O(k)$, $O(h)$ and $O(m \cdot f)$. Combining with this figure, we give the following proof.*

Suppose the transaction is $txn = tx(v_j, O_i \rightarrow O_{i+1})$ at B_t. For CCPT, $O(m)$ indicates that this transaction needs to be broadcast to the temporary committee (i.e., TC_l) in CC algorithm. $O(k)$ indicates the times that txn is transmitted by any node other than O_i in the l-th epoch, which can only happen after the txn is confirmed (i.e., after B_t) and before the next epoch, and the owner $O_k(k > i)$ of v_j needs to provide txn as a part of the proof when transferring v_j. $O(h)$ indicates the times that txn is transmitted by O_i after B_t when O_i wants to trade. As txn's abstract is included by ACC, all values held by O_i at this moment must provide txn as a part of their proof when they are transferred in the future (here implicit a weak assumption: O_i's all values will be transferred in the future). According to Appendix G, $O(m \cdot f)$ means txn is to be broadcast as a part of the prf in the future and needs to be broadcast in f future temporary committees. Note that in each temporary committee, the same transaction will only be transmitted at most m times. Therefore, txn only needs to be transmitted at most $m \cdot f$ times.

Using CCPT to measure system's scalability is more effective than just comparing TPS values. In Table 1, we present a simple analysis of several protocols' CCPT.

We make an informal but more intuitive comparison of the CCPT of each protocol here. Although the CCPT of EZchain consists of four items—m, k, h, and f, they will all stabilize at a constant level. The analysis is as follows: The size of the committee m generally has a certain upper bound, which does not expand indefinitely. The average transaction frequency of nodes is relatively fixed, thus the number of transactions in each epoch k is also fixed. The number of values held by a node h at a certain moment (for example, when a transaction occurs) is also maintained at a stable level (although the h of each node has a large variance, there is generally an upper bound for h). According to the analysis in Figure 4, the value of f is limited by many factors. An obvious upper bound of f is h, and generally f will be smaller than h. Therefore, EZchain's CCPT (i.e., $O(m)+O(k)+O(h)+O(m \cdot f)$) can be stabilized at a constant level, which is only related to the initialization parameters of the system, and is not related to the network size N.

As comparisons, the parameter b of VAPOR will continuously approach the network size N with the random walk of the transactions (for detailed analysis, please see Appendix F), that is, its CCPT value will still change with N. OmniLedger's parameter d represents the size of the shardings. As long as the number of shardings does not increase indefinitely, d must increase with the increase of N, thus OmniLedger's CCPT is still related to N. Algorand and Byzcoin use the committee mechanism while requiring the entire network to reach a consensus on all transactions. Therefore, their CCPT is equal to $O(m) + O(N)$.

And Theorem 4 also shows an important conclusion:

Corollary 1. EZchain's CCPT is not related to N and stable at a certain constant, thus, there is a positive linear correlation between EZchain's throughput and N.

According to the above analysis, as long as the initial parameters are determined (e.g., period T, number of initial values, frequency of transactions), EZchain's CCPT will stabilize to a certain constant after the system has been running for a period of time, and this constant is independent of N. Further, Here we give a simple model about throughput: when the node's processing power is sufficient,

$$BW_a = BW_t + BW_o,$$

where BW_a is the system's average available bandwidth per unit time, BW_t is the average bandwidth for transmitting transactions per unit time and BW_o is the average other bandwidth per unit time. And $BW_t = CCPT_{EZchain} \times throughput$, BW_o consists of and the bandwidth consumed by AC and CC. BW_a, BW_o and N have positive linear correlations. ACC and CCC need to be recorded by all nodes, however, the proportion of resources occupied by ACC and CCC is much smaller than that of BW_t, because ACC and CCC are so-called "beacon chain", which only records transactions' abstracts and some additional information. If the size ratio of the beacon chain to the complete information is $1 : 9$, then when BW_a is increased by 10 units, for the best efficiency, it should be allocated as follows: BW_o is increased by 1 unit and BW_t is increased by 9 units. Namely, most of the increase in BW_a is allocated to BW_t, and CCPT is not related to N, thus the throughput of EZchain can almost increase linearly with the expansion of network size N.

However, VAPOR does not have this feature (see Appendix F for details). In addition, reasonable selection of the values for the transactions can make EZchain's CCPT further reduced (see Appendix B.1 for details).

5.2.2 Transaction's confirmation latency

In EZchain, the steps of the transaction $txn = tx(v_i, x \rightarrow y)$ include: x sends proof p_1 to y to indicate that x is currently the legal owner of v_i (time consuming: t_1); y verifies p_1's legality (time consuming: t_2); x submits txn's abstract with its inf to the ACC and waits for its confirmation (time consuming: t_3); x sends p_2 to y to indicate that the new owner of v_i is y (time consuming: t_4); y verifies p_2's legality (time consuming: t_5).

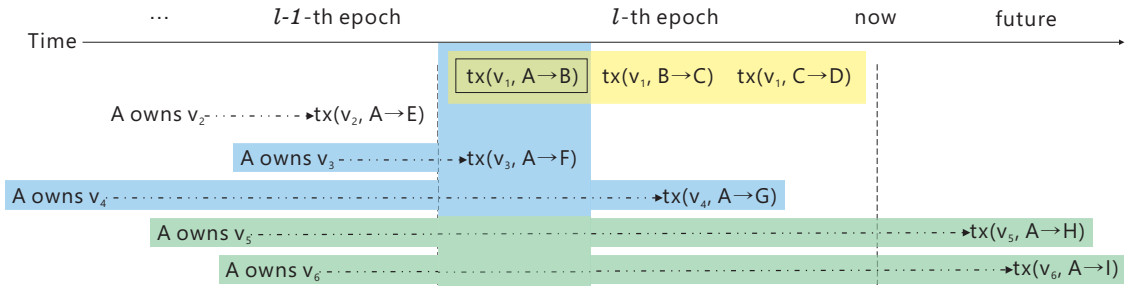

Fig. 4. A CCPT's analysis of the transaction $tx(v_1, A \to B)$. The yellow part indicates the communication cost represented by $O(k)$, the blue and green parts stand for the communication cost represented by $O(h)$, and the green part also indicates the communication cost represented by $O(m \cdot f)$. The yellow part shows that after the transaction $txn = tx(v_1, A \to B)$, all v_1's transfers in the l-th epoch need to transmit txn as a part of the proof. The blue and green parts mean that as long as the "spans" of the values v_3, v_4 and v_5 owned by A (i.e., the time from A owns it to A transfers it) contain txn, then when these values are eventually transferred, txn must be transmitted as a part of proof as well. Furthermore, the green part also means that v_5 and v_6 will be transferred in the future, and A must provide the corresponding prf for their transfers in the future temporary committees (see Appendix G, since this must be the first time they are transferred in the future committees, prf should be broadcast in this temporary committee), and prf must contain txn. f is the number of the values like v_5, v_6 (more precisely, the number of future temporary committees they correspond to, since v_5 and v_6 may be transferred by A in the same future epoch).

TABLE 1
The complexity of different protocols' CCPT. Here m represents the size of the committee in each protocol, N stands for the number of the whole network's nodes, k is the average number of times of the value's transfers in one EZchain's epoch, h is the average number of values owned by an EZchain node when a transaction is confirmed, f indicates how many future EZchain temporary committees that need to reach consensus on a transaction, b denotes the average transaction blocks of each state acquired by a node in VAPOR, and d is the shard's size in OmniLedger.

Protocol	EZchain	VAPOR	OmniLedger	Algorand	Ouroboros	Byzcoin	Bitcoin-NG
CCPT	$O(m)+O(k)+O(h)+O(m \cdot f)$	$O(b)$	$O(d)$	$O(m)+O(N)$	$O(N)$	$O(m)+O(N)$	$O(N)$

Since steps $1+2$ and 3 can be performed simultaneously, node x only needs to wait for $\max(t_1+t_2, t_3)+t_4+t_5$ at most. In the actual situation, $t_3 > t_1+t_2 \gg t_4+t_5$ (Bitcoin's latency is about 60 minutes [36], Ouroboros and Algorand's latencies are about 1 minute [10], [13]). Therefore, the transaction's confirmation latency of our protocol depends almost exclusively on the consensus latency of the AC algorithm.

5.2.3 Storage cost

In our protocol, the information that needs to be stored by node x is as follows: the complete ACC, i.e., $\mathcal{B}$; the complete CCC, i.e., $\mathcal{CB}$; the x's necessary transactions, i.e., $\mathcal{PB}$; all the proofs of the values that x owns, denoted as $\mathcal{PV}$. Note that $\mathcal{PB} \cap \mathcal{PV} \neq \varnothing$, and the complete storage of the $\mathcal{PB}$ is not a necessary condition for the node (if the user wants to record all his transaction records, the complete $\mathcal{PB}$ needs to be stored, yet in terms of the main function of EZchain, the complete $\mathcal{PB}$ is not necessary). Thus the two cases about $\mathcal{PB}$ will be discussed in the following.

Store the complete $\mathcal{PB}$ We use $M(\cdot)$ to denote the storage needed by x to store a certain data, and use $M(*)$ to denote all the storages necessary for the node x. In EZchain, $M(*) = M(\mathcal{B}) + M(\mathcal{CB}) + M(\mathcal{PB}) + (M(\mathcal{PV}) - M(\mathcal{PB} \cap \mathcal{PV}))$. So the expectation of the above formula is $E(M(*)) = EM_{PB} + EM_B + EM_{CB} + n_v \cdot (p \cdot 0 + (1-p) \cdot \kappa_{\frac{T}{2}})$, where $EM_{PB} = E(M(\mathcal{PB}))$, $EM_B = E(M(\mathcal{B}))$, $EM_{CB} = E(M(\mathcal{CB}))$, n_v is the average number of values held by a node, p is the probability of including at least one checkpoint within the time span of the value owned by x, and $\kappa_{\frac{T}{2}}$ is the average number of transactions of a node during the time of $T/2$. We list various protocols' $E(M(*))$ in Table 2 (See Appendix C for specific analysis).

Then we have $(1-p) \cdot \kappa_{\frac{T}{2}} < \kappa_{\frac{T}{2}} < \kappa_{A_v}$, and in most cases, $(1-p) \cdot \kappa_{\frac{T}{2}} \ll \kappa_{A_v}$. Moreover, EM_{CB} is negligible compared to other items. So VAPOR's $E(M(*))$ is further optimized by our protocol.

Optimal storage In order to save storage space, the user can choose to store only the information necessary for the protocol, i.e., the beacon chain (ACC and CCC) and the proof of the value held. According to the model in Subsection 3.1, when all initialization parameters are determined (e.g., the trading frequency of the node is a constant, denoted as f_t) and the system is stable for a period of time, then the expectation of the number of values held by the node at any time should be constant n_v (since the f_t of each node is the same, namely, the node receives and sends a value with the same frequency, the expectation of n_v is constant); suppose that the expectation of the time of each value held by the same node is t_n, t_n represents the average length of time a value is held by a node, then as long as n_v and f_t are fixed, t_n is also constant. Then the necessary (optimal) storage of EZchain node is $EM_B + EM_{CB} + \max\{O(Cn_v t_n f_t), O(Cn_v T f_t)\}$, where C is the constant representing the additional proof required by the held value because of the transactions (i.e., each time a node makes a transaction, the value it holds must add this transaction to the proof set), and T is the epoch's period. This analysis shows that EZchain's optimal storage strategy can ensure that the expectation of node's non-beacon chain storage cost is stable to a constant, and this constant does not change with N or its transactions.

5.2.4 Fairness

The fairness is defined as the proportion of blocks "mined" by different nodes that reach the threshold spec-

TABLE 2
The storage requirements for the normal node in different protocols, where A_v is the average age of a value (i.e., the time since its birth), EM_{sb} is the storage required for the state block [21], Tr is the storage occupied by all transactions, Tr_e is the average storage occupied by the transactions in an epoch of OmniLedger, S is the number of shards, and $\gamma = 1.3$ (in Algorand, for 1 MByte blocks, this would be a 30% storage overhead for the block certificates [10]).

Protocol	EZchain	VAPOR	OmniLedger	Algorand	Byzcoin	Bitcoin-NG
$E(M(*))$	$EM_{PB} + EM_B + EM_{CB} + (1-p) \cdot n_v \kappa_{\frac{T}{2}}$	$EM_{PB} + EM_B + n_v \kappa_{A_v}$	$EM_{PB} + EM_{sb} + \frac{Tr_e}{S}$	$EM_{PB} + \gamma \cdot \frac{Tr}{S}$	Tr	Tr

ified by the consensus algorithm. Thus EZchian's fairness is the same as that of the AC algorithm since the probability of mining blocks only depends on the leader election, which is only related to the AC algorithm, and the changes on $\mathcal{S}_A$ (the original AC algorithm) in EZchain do not affect the result's probability as well (it just changes the main chain to the abstract chain).

5.3 Optimization

After the above analysis, some improvements can still help EZchain improve its performance. We give some specific optimization schemes in Appendix B:

- Choose the appropriate values for the transactions (Appendix B.1).
- Choose the appropriate epoch's span T (Appendix B.2).
- Save ACC's storage space (Appendix B.3).
- Save the bandwidth resources consumed by ACC (Appendix B.4).

6 Evaluation

In this section, we evaluate the performance of our protocol by instantiating a lightweight EZchain. We focus on the following aspects:

- Investigate the trends of EZchain's CCPT under different parameter settings (network size, committee's size, epoch's period, etc). And further verify whether EZchain satisfies Corollary 1.
- Examine the storage costs (i.e., ACC, CCC, transactions, and proofs) of the nodes in EZchain, and assess their trends under different settings.
- Evaluate the relationship between EZchain's security and its CCPT.

6.1 Experiment Setup

The upper bound of network bandwidth is a direct influence factor of TPS, and different network structures, network bandwidth and network delay will greatly affect the experimental results and evaluation of protocol performance. Therefore, the experiments of some protocols have abandoned the investigation of TPS, and are more concerned with bandwidth resources that can be used to generate throughput (e.g., OHIE [24]). CCPT helps us solve the problem of network bandwidth setting. Compared with complex network settings, CCPT only considers the number of times that a transaction is transmitted, and does not even need to consider the content and size of the transaction. And the conclusions drawn from the analysis of CCPT holds under any network bandwidth condition (CCPT does not change as the network size and the bandwidth resources' changes).

Moreover, the lightweight simulation based on CCPT allows us to expand the simulated P2P network size up to $100,000$ nodes with a limited experimental environment. In the lightweight prototype, we omit some components of EZchain (e.g., AC algorithm, digital signature process), which do not affect the simulation results. The following experiment mainly tracks all the transaction trajectories during the execution of EZchain and other various protocols, and records each transfer to count the final CCPT value. We also record the complete ACC, CCC, and local storage content of each node to evaluate the performance of EZchain and other protocols in terms of storage costs.

We preset some values: the size of each transaction is $500\ kb$ (this is the same as Bitcoin); and we conservatively estimate that each abstract $A_i(x)$'s size is $200\ bytes$, and each CC block is $1\ MB$ in size. Besides, the parameters that we focus on are: network size N, committee's size M, the number of values n_v held by each node at initialization (note that there is no need to worry about the increase of the number of values, since the value's merging operation can be used to maintain it at a standard level), the node's trade frequency k (i.e., the number of transactions generated/processed by a node per unit time), and the period T of the epoch.

For the AC algorithm, we use a completely random selection of leaders for each round, and simulate the spacing between the leaders with a normal distribution.

In addition, all the results in the evaluation are verified by at least 5 experiments. And since EZchain and VAPOR's transactions will be transmitted in the uncertain future, all of our experiments have run enough rounds to get stable and correct CCPT data.

6.2 EZchain's Scalability

Based on the previous analysis, in this experiment we only change N and observe the trend of CCPT. The results of six different experiments are presented in Figure 5, in which we reasonably fix other parameters: $M = 200$ (this reaches a security level higher than OmniLedger), $n_v = 10$, $k = 0.01$ and $T = 2000$. It can be clearly observed from Figure 5 that EZchain's CCPT remains at around 800 regardless of N, and this also verifies the Corollary 1.

Next we concentrate on the relationship between EZchain's CCPT and other parameters: n_v and T. This experimental setting adjusts N to 1000. The results are presented in Figures 6 and 7, respectively. The CCPT is linearly

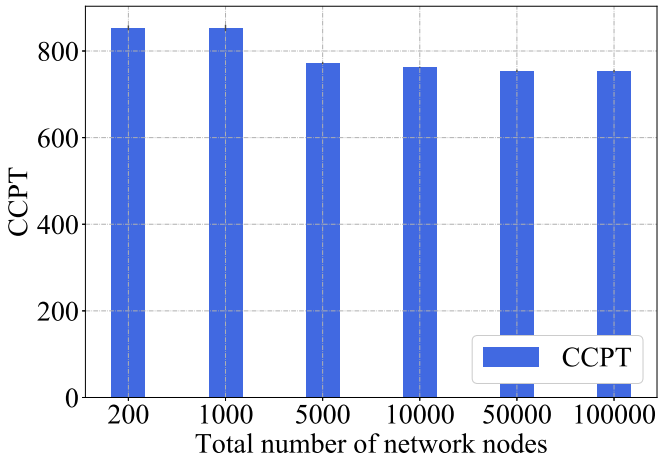

Fig. 5. Relationship between EZchain's CCPT and network size N.

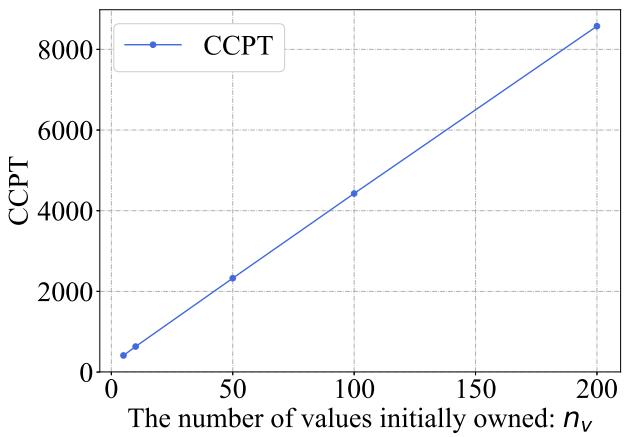

Fig. 6. Relationship between the CCPT of EZchain and n_v.

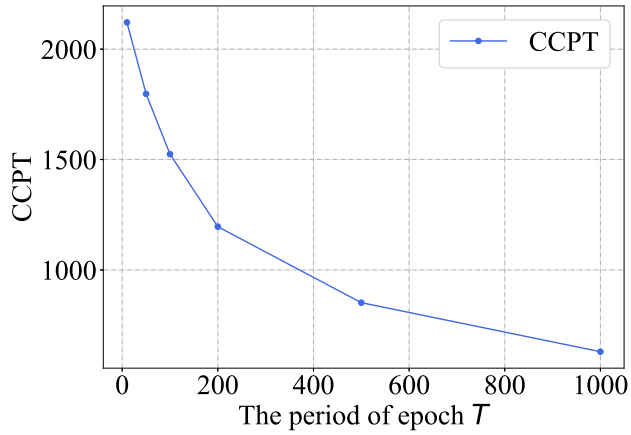

Fig. 7. Relationship between the CCPT of EZchain and T.

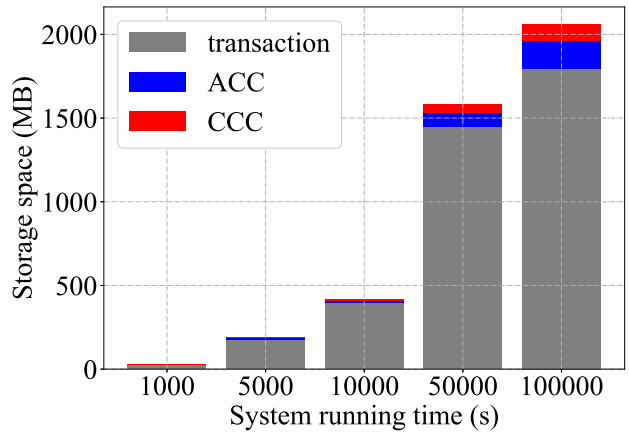

Fig. 8. Relationship between the storage cost (MB) of EZchain and the system's running time (s).

related to n_v, which suggest that we should choose as few initial values as possible when implementing EZchain.

Yet there is negative correlation between CCPT and T, this is because T becomes larger and the bandwidth consumption of the CC algorithm is "diluted". In fact, we predict that as T continues to increase, CCPT will be positively correlated with T, since when T is large enough, EZchain is equivalent to the VAPOR protocol (see Appendix F for the analysis of VAPOR's CCPT). However, due to the limitations of the experimental equipment, we do not complete the larger situation of T.

6.3 EZchain's Storage Cost

In order to optimize the performance of EZChain, we adopt the optimal storage strategy (the optimal storage in Subsection 5.2.3) in this experiment. Here we mainly investigate the trend of EZchain node's storage cost with the increase of the system's running time and the network size (note that the increase in running time leads to a growing number of transactions). And the experimental results are shown in Figures 8 and 9.

According to Figure 8, whose experimental settings are: $N = 1000$, $n_v = 100$, $T = 1000$, ACC and CCC's cost are linearly related to the running time, while storage consumed by transactions grows slowly. More precisely, the storage required by transactions will converge to a constant with increasing running time, which is revealed by the smaller experimental network environment ($N = 100$, $n_v = 5$, $k = 0.1$, $T = 100$) in Figure 10. This result also validates the previous analysis of the optimal storage strategy for EZchain. A simple explanation for this result is: as the system runs, the node stores new transactions and also continually discards old transactions that are no longer needed, the two trends will eventually reach a balance.

As shown in Figure 9, EZchain's transaction storage cost has a good attribute that it is independent of the network size, which is a merit that were not achieved by traditional sharding protocols. However, the storage of ACC is still linearly related to N (for this problem, we give a feasible solution in Appendix B.3).

In summary, under fixed parameter settings (i.e., n_v, T, k), the non-beacon information (i.e., in addition to ACC and CCC) of EZchain node requires a fixed amount of storage cost, which is independent of running time and network size. In contrast, other sharding protocols fail to maintain the original storage requirements as the network grows larger, unless they add shardings, which however increases the burden on the entire network.

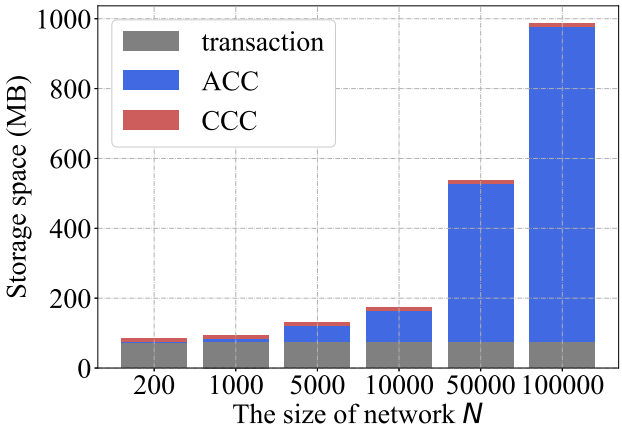

Fig. 9. Relationship between the storage cost (MB) of EZchain and N.

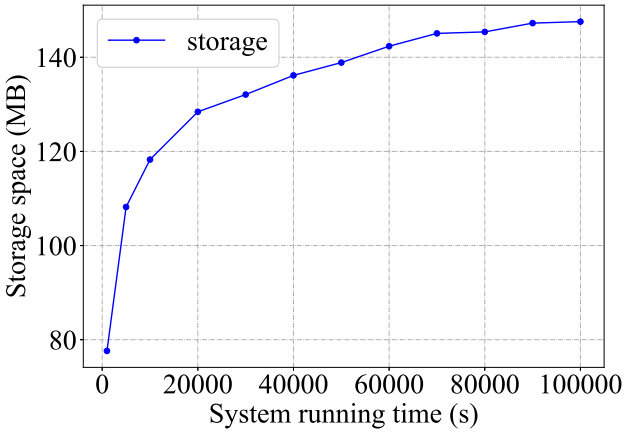

Fig. 10. EZchain node's transaction storage cost (MB) changes with the system's running time (s).

6.4 The relationship between scalability and security of EZchain

The security and scalability of EZchain are both related to the temporary committee's size M. As shown in Figure 11 (the experimental setting is $N = 1000$, $n_v = 10$, $T = 1000$), high security and high throughput are not compatible. However, EZchain has excellent throughput under the premise of ensuring reasonable security (the dotted line in the figure indicates the security level of OmniLeger, and EZchain's CCPT is only about 600 at this time).

6.5 Comparative experiments with other protocols

In this subsection, we focus on the performance comparison between EZchain and other mainstream protocols. Specifically, we choose Algorand, OmniLeager, and Vapor as the comparison objects, which respectively correspond to the mainstream consensus algorithm, sharding algorithm, and the first VCB protocol. The other protocols in this experiment all use the unified parameter settings as in Subsection 6.1.

The experimental results of CCPT are shown in Fig. 12. In general, EZchain has obvious advantages over the other three protocols in terms of CCPT. After the system has been running stably for a long enough time, the CCPTs

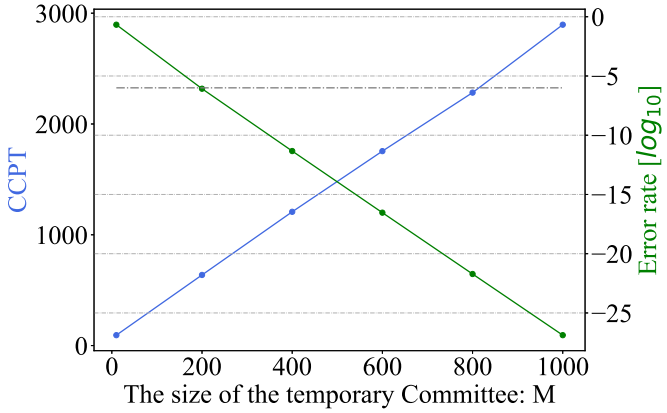

Fig. 11. Trend of EZchain's security and CCPT with the increase of M.

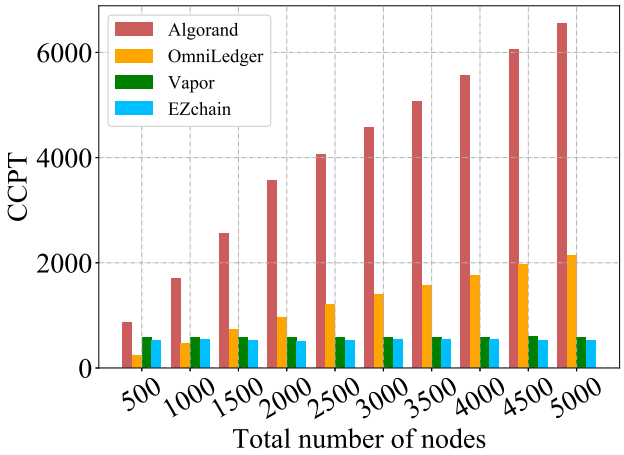

Fig. 12. Comparative experiments of EZchain, Vapor, Omniledger and Algorand's CCPT.

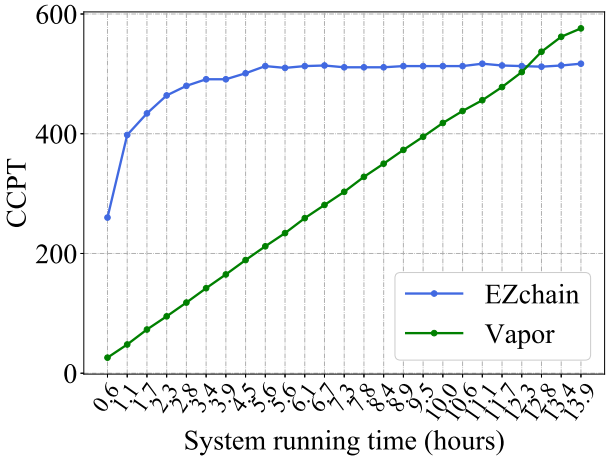

Fig. 13. Trend of EZchain and Vapor's CCPT with the system running time.

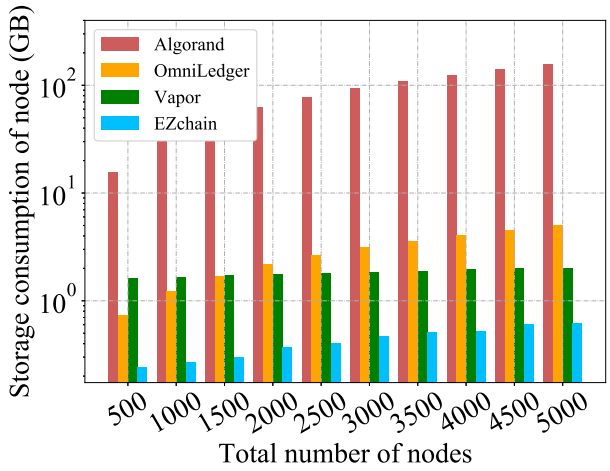

Fig. 14. Comparative experiments of EZchain, Vapor, Omniledger and Algorand's node storage cost.

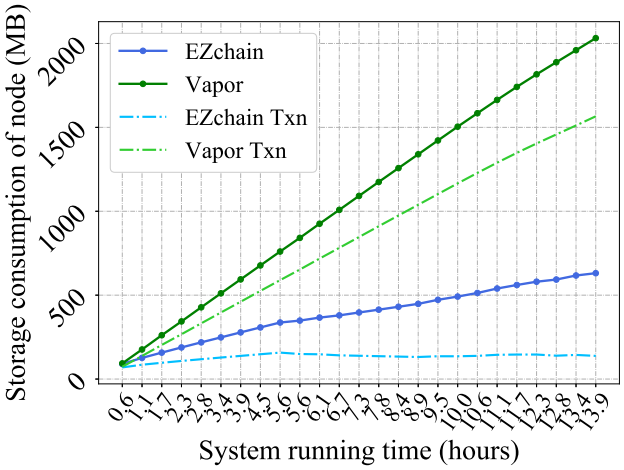

Fig. 15. Trend of EZchain and Vapor's storage cost with the system running time, where Txn represents the space used to store transactions.

of Algorand and OmniLeager have an obvious linear relationship with the network size N (i.e., the total number of nodes), while EZchain and Vapor's CCPTs are almost constant values. In a 5000 nodes network, the EZchain's CCPT is about 4.1 times and 12.6 times lower than that of OmniLeager and Algorand, respectively.

In addition, our experiments also show that the Vapor's CCPT increases with the increase of system running time. As shown in Fig. 13, the CCPT of EZchain can quickly stabilize to a constant level within a period of time, while the CCPT of Vapor is almost linearly related to the running time.

In terms of storage cost, Fig. 14 shows the comparison results [2]. Compared with OmniLeager and Algorand, EZchain has obvious advantages in both the node storage cost value and the growth rate. In the largest experimental network, the EZchain node saves about 252 times of storage space than that of Algorand node. The storage cost growth rates of EZchain and Vapor are almost the same [3]. However, as shown in Fig. 15, the Vapor node's storage cost continues to increase with the running of the system, and is significantly greater than that of EZchain. The transaction storage space of the EZchain node will stabilize at around a constant value, but Vapor cannot achieve this.

7 Related Work

Nakamoto Consensus (NC) and Byzantine Agreement (BA) The NC-based blockchain protocols are represented by Bitcoin, e.g., NC-max [37], Bitcoin-NG [11]; and the BA-based blockchain protocol hands over consensus task to the BFT scheme, e.g., Byzcoin [12]. Subsequently, Algorand [10], Ouroboros [13] and other protocols [14], [15] used Verifiable Random Function (VRF) [20] with PoS to replace the original "mining" to complete the leader election, resulting in enhanced performance. These protocols have strict security models and security proofs, and the AC algorithms of our protocol also need to be selected from these protocols to satisfy the requirements of ACC. However, in terms of practicality: Bitcoin's limit of 7 TPS, large network environment's constraints on BFT. And most importantly, all nodes need to record the global ledger, i.e., CCPT= $O(N)$.

Directed Acyclic Graph-type (DAG) protocols Such protocols include Phantom [16], Conflux [17] and IOTA [18]. They replace the chain structure with DAG, i.e., the new block can connect multiple parent blocks. This structure can better avoid bandwidth waste, but the structure of the DAG also brings some problems: the transactions cannot be sequenced; and the transactions are repeatedly submitted. This makes it difficult to strictly prove the security of the DAG-type protocols or requires additional security assumptions. And the DAG-type protocols' CCPT are still $O(N)$.

VAPOR The VAPOR protocol [22] and its VCB framework [38], [39], [40] directly inspires our protocol. Although VAPOR satisfies its CCPT= $o(N)$, there is still a difference between its scalability and the traditional sharding protocol. The increase of N will increase the upper bound of VAPOR's CCPT, and as the running time increases, its CCPT will increasingly approach this upper bound. VAPOR also has some advantages beyond traditional sharding, such as stronger security and simpler protocol content. EZchain was born by combining the advantages of the two.

Sharding technique Sharding [21], [23], [24], [25] is an effective solution to improve system's scalability. Omniledger [21] is the first so-called "scale-out" sharding protocol, which can tolerate 1/4 malicious nodes without permission operation, and relevant experimental data show that Omniledger supports Visa-level workloads and beyond. OHIE [24] aims at brevity, and it can compose as many parallel instances of Bitcoin's original (and simple) backbone protocol as needed to achieve an excellent throughput. Rapidchain [25], the first sharding-based public blockchain protocol, is resilient to Byzantine faults from up to a 1/3 fraction of its participants. Monoxide [41] proposes asynchronous consensus zones and divides the whole network into parallel zones, which scales the blockchain system linearly without compromising decentralization or security.

2. Due to the large difference between the experimental data of Algorand and EZchain, we use the exponential ordinate.

3. Due to the compression of the exponential ordinate, the histogram of Vapor in Fig. 14 has almost no growth in vision.

Many sharding protocols divide the network into several subsets and execute consensus protocols in them. Sharding achieves better scalability and decentralization through "divide and rule", yet it still faces many problems: frequent cross-shard transactions lead to a drop in efficiency; the original security assumptions do not hold in the shards, which may introduce new security issues (e.g., there is no guarantee that the number of failure nodes in each shard is still less than $1/3$ of the total). By contrast, the passive sharding of EZchain does not generate cross-shard transaction, and has the same security assumptions as the AC algorithm and provable security.

Off-chain technique The data structure of EZchain and VAPOR is similar to the off-chain protocol RSK [30], however EZchain is essentially a sharding scheme. The off-chain protocols [30], [33], [42] treat the main chain as a trusted third party and mortgages the deposit to complete the transaction. It aims to reduce the consensus bandwidth resources through off-chain trading, which does not need to reach consensus in the whole network. RSK [30] provides a platform pegged to Bitcoin and allows Bitcoin miners to participate in the smart contract by rewarding them via merge-mining. Plasma [33] has many sidechains attached to the Ethereum main chain. These sidechains can run as independent blockchains and can have their consensus mechanisms. Lightning Network [42] is a typical off-chain solution that allows two users to send nearly unlimited payments off-chain by using a payment channel. Yet, the security of the off-chain transactions cannot be guaranteed by the main chain's protocol, and the transactions in EZchain do not require any form of collateral.

Solutions related to Block Data The Segregated Witness [43] removes the unlocking signatures from the original transaction hashes, which can slightly increases the scalability of blockchian. Bitcoin-Cash [44] scales the blockchain by increasing block capacity directly. However, the block size is limited by the network bandwidth, and larger blocks may lead to the centralization problem [45]. Block compression can achieve the same effect and also reduce storage overhead. Compact block proposed by BIP-152 [46] only contains the header of the block and some short transaction IDs (TXIDs) and receivers need to reconstruct the full block by comparing TXIDs of the transactions in their Mempool.

8 Conclusion

In this paper, we presented EZchain, which is the first passive sharding blockchain protocol with constant CCPT. It inherits from and improves the VCB framework with proven validity, security and liveness. Compared to traditional sharding, EZchain provides a simple implementation, original security assumption and higher security. It does not generate cross-shard transactions, and requires no additional anti-Sybil attack verification and network sharding algorithm. In addition, we propose a new scalability measurement mechanism based on the transaction's transmission. In the experiments with up to 100,000 nodes, EZchain is verified to have constant CCPT, and unlike other sharding protocols, the non-beacon storage cost of EZchain node is independent of the network size and the system's running time. Comparative experiments with other protocols have also verified the above points.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Consulted*, 2008.

[2] J. A. Garay, A. Kiayias, and N. Leonardos, "The bitcoin backbone protocol: Analysis and applications," in *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*, 2015, pp. 281–310. [Online]. Available: https://doi.org/10.1007/978-3-662-46803-6_10

[3] A. Kiayias and G. Panagiotakos, "Speed-security tradeoffs in blockchain protocols," *IACR Cryptology ePrint Archive*, vol. 2015, p. 1019, 2015. [Online]. Available: http://eprint.iacr.org/2015/1019

[4] J. A. Garay, A. Kiayias, and N. Leonardos, "The bitcoin backbone protocol with chains of variable difficulty," in *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part I*, 2017, pp. 291–323. [Online]. Available: https://doi.org/10.1007/978-3-319-63688-7_10

[5] L. Kiffer, R. Rajaraman, and A. Shelat, "A better method to analyze blockchain consistency," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, 2018, pp. 729–744. [Online]. Available: https://doi.org/10.1145/3243734.3243814

[6] K. Croman, C. Decker, I. Eyal, A. E. Gencer, A. Juels, A. E. Kosba, A. Miller, P. Saxena, E. Shi, E. G. Sirer, D. Song, and R. Wattenhofer, "On scaling decentralized blockchains - (A position paper)," in *Financial Cryptography and Data Security - FC 2016 International Workshops, BITCOIN, VOTING, and WAHC, Christ Church, Barbados, February 26, 2016, Revised Selected Papers*, 2016, pp. 106–125. [Online]. Available: https://doi.org/10.1007/978-3-662-53357-4_8

[7] A. Beikverdi and J. Song, "Trend of centralization in bitcoin's distributed network," in *16th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing, SNPD 2015, Takamatsu, Japan, June 1-3, 2015*, 2015, pp. 377–382. [Online]. Available: https://doi.org/10.1109/SNPD.2015.7176229

[8] de Vries and Alex, "Bitcoins̈ growing energy problem," vol. 2, no. 5, pp. 801–805, 2018.

[9] A. E. Gencer, S. Basu, I. Eyal, R. van Renesse, and E. G. Sirer, "Decentralization in bitcoin and ethereum networks," *CoRR*, vol. abs/1801.03998, 2018. [Online]. Available: http://arxiv.org/abs/1801.03998

[10] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling byzantine agreements for cryptocurrencies," in *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*, 2017, pp. 51–68. [Online]. Available: https://doi.org/10.1145/3132747.3132757

[11] I. Eyal, A. E. Gencer, and R. V. Renesse, "Bitcoin-ng: a scalable blockchain protocol," in *Usenix Conference on Networked Systems Design and Implementation*, 2016.

[12] E. Kokoriskogias, P. Jovanovic, N. Gailly, I. Khoffi, L. Gasser, and B. Ford, "Enhancing bitcoin security and performance with strong consistency via collective signing," *Applied Mathematical Modelling*, vol. 37, no. 8, pp. 5723–5742, 2016.

[13] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part I*, 2017, pp. 357–388. [Online]. Available: https://doi.org/10.1007/978-3-319-63688-7_12

[14] I. Bentov, R. Pass, and E. Shi, "Snow white: Provably secure proofs of stake," *IACR Cryptology ePrint Archive*, vol. 2016, p. 919, 2016. [Online]. Available: http://eprint.iacr.org/2016/919

[15] T. Hanke, M. Movahedi, and D. Williams, "DFINITY technology overview series, consensus system," *CoRR*, vol. abs/1805.04548, 2018. [Online]. Available: http://arxiv.org/abs/1805.04548

[16] Y. Sompolinsky and A. Zohar, "PHANTOM: A scalable blockdag protocol," *IACR Cryptology ePrint Archive*, vol. 2018, p. 104, 2018. [Online]. Available: http://eprint.iacr.org/2018/104

[17] S. Xiao, X. A. Wang, and H. Wang, "Large-scale electronic voting based on conflux consensus mechanism," in *Innovative Mobile and Internet Services in Ubiquitous Computing - Proceedings of the 13th International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing (IMIS-2019), Sydney, NSW, Australia, 3-5 July 2019.*, 2019, pp. 291–299. [Online]. Available: https://doi.org/10.1007/978-3-030-22263-5_28

[18] S. Popov, O. Saa, and P. Finardi, "Equilibria in the tangle," *CoRR*, vol. abs/1712.05385, 2017. [Online]. Available: http://arxiv.org/abs/1712.05385

[19] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*, 1999, pp. 173–186. [Online]. Available: https://dl.acm.org/citation.cfm?id=296824

[20] L. Kohl, "Hunting and gathering: Verifiable random functions from standard assumptions with short proofs," *IACR Cryptology ePrint Archive*, vol. 2019, p. 42, 2019. [Online]. Available: https://eprint.iacr.org/2019/042

[21] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omniledger: A secure, scale-out, decentralized ledger via sharding," in *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*, 2018, pp. 583–598. [Online]. Available: https://doi.org/10.1109/SP.2018.000-5

[22] Z. Ren and Z. Erkin, "VAPOR: a value-centric blockchain that is scale-out, decentralized, and flexible by design," *CoRR*, vol. abs/1810.12596, 2018. [Online]. Available: http://arxiv.org/abs/1810.12596

[23] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, 2016, pp. 17–30. [Online]. Available: https://doi.org/10.1145/2976749.2978389

[24] H. Yu, I. Nikolic, R. Hou, and P. Saxena, "OHIE: blockchain scaling made simple," *CoRR*, vol. abs/1811.12628, 2018. [Online]. Available: http://arxiv.org/abs/1811.12628

[25] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, 2018, pp. 931–948. [Online]. Available: https://doi.org/10.1145/3243734.3243853

[26] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A sharded smart contracts platform," in *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*, 2018. [Online]. Available: http://wp.internetsociety.org/ndss/wp-content/uploads/sites/25/2018/02/ndss2018_09-2_Al-Bassam_paper.pdf

[27] J. R. Douceur, "The sybil attack," in *Peer-to-Peer Systems, First International Workshop, IPTPS 2002, Cambridge, MA, USA, March 7-8, 2002, Revised Papers*, 2002, pp. 251–260. [Online]. Available: https://doi.org/10.1007/3-540-45748-8_24

[28] Z. Ren, K. Cong, T. Aerts, B. de Jonge, A. Morais, and Z. Erkin, "A scale-out blockchain for value transfer with spontaneous sharding," in *Crypto Valley Conference on Blockchain Technology, CVCBT 2018, Zug, Switzerland, June 20-22, 2018*, 2018, pp. 1–10. [Online]. Available: https://doi.org/10.1109/CVCBT.2018.00006

[29] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, "Hotstuff: BFT consensus with linearity and responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019.*, 2019, pp. 347–356. [Online]. Available: https://doi.org/10.1145/3293611.3331591

[30] S. D. Lerner, "Rsk white paper updated," https://www.rsk.co/wp-content/uploads/2019/02/RSK-White-Paper-Updated.pdf, january 29, 2019.

[31] G. Wood, "Polkadot: Vision for a heterogeneous multi-chain framework," *White Paper*, 2016.

[32] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," 2016.

[33] J. Poon and V. Buterin, "Plasma: Scalable autonomous smart contracts," https://plasma.io/plasma.pdf, august 11, 2017.

[34] M. Andrychowicz and S. Dziembowski, "Pow-based distributed cryptography with no trusted setup," in *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part II*, 2015, pp. 379–399. [Online]. Available: https://doi.org/10.1007/978-3-662-48000-7_19

[35] R. Pass and E. Shi, "Hybrid consensus: Efficient consensus in the permissionless model," in *31st International Symposium on Distributed Computing (DISC 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.

[36] BitcoinWiki, "Confirmation," https://en.bitcoin.it/wiki/Confirmation, april, 2019.

[37] Nervos, "Nervos ckb's consensus agreement nc-max: Breaking through the limits of nakamoto consensus' throughput," https://talk.nervos.org/t/nervos-ckb-nc-max-nakamoto-consensus/1295, february 14, 2019.

[38] D. Chaum, "Blind signatures for untraceable payments," in *Advances in Cryptology: Proceedings of CRYPTO '82, Santa Barbara, California, USA, August 23-25, 1982.*, 1982, pp. 199–203.

[39] D. Chaum, A. Fiat, and M. Naor, "Untraceable electronic cash," in *Advances in Cryptology - CRYPTO '88, 8th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1988, Proceedings*, 1988, pp. 319–327. [Online]. Available: https://doi.org/10.1007/0-387-34799-2_25

[40] T. Okamoto and K. Ohta, "Universal electronic cash," in *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*, 1991, pp. 324–337. [Online]. Available: https://doi.org/10.1007/3-540-46766-1_27

[41] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *NSDI*. USENIX Association, 2019, pp. 95–112.

[42] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," https://lightning.network/lightning-network-paper.pdf, january 14, 2016.

[43] E. Lombrozo, J. Lau, and P. Wuille, "Segregated witness (consensus layer)," *Bitcoin Core Develop. Team, Tech. Rep. BIP*, vol. 141, 2015.

[44] Y. Kwon, H. Kim, J. Shin, and Y. Kim, "Bitcoin vs. bitcoin cash: Coexistence or downfall of bitcoin cash?" in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 935–951.

[45] en.bitcoin.it, "Block size limit controversy," https://en.bitcoin.it/wiki/Block_size_limit_controversy.

[46] M. C. ¡bip152@bluematt.me¿, "Bip: 152 (compact block relay)," https://github.com/bitcoin/bips/blob/master/bip-0152.mediawiki.

APPENDIX A
PROOF

A.1 Proof of Theorem 1

Proof [Theorem 1] *The following proof is based on the premise: the TC_l does not fail (i.e., the Byzantine nodes in TC_l do not exceed half of the total since the legal CB_l needs to contain more than $1/2$ of the TC_l members' collective signatures, and the failure probability is exponentially small, see Appendix D for details).*

The asynchronous consistency of CCC: For any legal txn (i.e., txn's abstract conforms to ACC), any two different honest nodes x and y will not have different viewpoints on txn, since the asynchronous consistency and immutability of ACC leads to the uniqueness of txn. As for any proof p (whether included in the prf or submitted by the node), due to the correctness and certainty of **GetOwner** *(theorem 2), the proof p that can make txn legal is unique, thus x and y will not have different viewpoints on legal p as well. In summary, all information affecting $\mathcal{CB}$ is determined and unique in the viewpoints of any different honest nodes, therefore the consistency of $\mathcal{CB}$ is guaranteed.*

The synchronous liveness of CCC: Based on the synchronous assumption for the honest node's transmission delay in Section 3, τ_1, τ_2, τ_3 and τ_4 in the CCC algorithm are constants, and their values are related to the synchronous assumption. And the leader election in TC_l will succeed in the end (according to the premise, at least half of the members in TC_l are honest), thus the synchronous liveness is established in CCC.

The correctness of CCC: First, we introduce a premise: for any legal transaction $txn = tx(v, A \rightarrow B)$ in B_k, both A and B must have proofs p_1 and p_2, which makes **GetOwner**$(v, B_{k-1}, p_1) = A$, **GetOwner**$(v, B_k, p_2) = B$ *(the premise is naturally explained in Algorithm 7). Then for any submitted transaction $txn = tx(v, x \rightarrow y)$ in B_k, if at least one of x and y is honest (without loss of generality, let x be honest), then txn must be legal. So in the first round of broadcasting CB_l, if x finds the txn's index $\in CB_l$, then x will definitely submit the legal $p_1 =$***Proof***(v, B_{k-1}) and $p_2 =$***Proof***(v, B_k), and due to the asynchronous consistency and immutability of ACC, thus all factors (i.e., proof, ACC) affecting* **GetOwner***'s results have not changed, i.e., all honest nodes in TC_l agree with* **GetOwner**$(v, B_{k-1}, p_1) = x$ *and* **GetOwner**$(v, B_k, p_2) = y$. *Therefore the index of txn must not appear in the final CB_l. Secondly, due to the correctness of* **GetOwner** *(see Section 4.4, Theorem 2), there is no transaction without legal proof that can pass the test. Therefore, the correctness of CCC holds.*

A.2 Proof of Theorem 2

Proof [Theorem 2] *First, if p is a complete proof, then we use induction. Assume that O_{i-1} transfers v_j to O_i at $B_{k'+1}$ with a legal proof p'. Now O_i wants to transfer v_j to O_{i+1} at B_{k+1}, O_i submits $tx(v_j, O_i \rightarrow O_{i+1})$ to B_{k+1}, as long as O_i is an honest node, then O_i must have p and p'' s.t.* **GetOwner**$(v_j, B_k, p) = O_i$ *and* **GetOwner**$(v_j, B_{k+1}, p'') = O_{i+1}$, *where $p = p' \cup \{txs_u(O_i) \mid k'+1 \leq u \leq k\} \cup \{O_i\text{'s public key}\}$ and $p'' = p \cup \{txs_{k+1}(O_i)\}$, since p' is legal and O_i does not make double-spending attacks (double-spending attacks can be checked by* **GetOwner***), p and p'' are both legal. And these proofs can be provided independently by O_i (p' is transmitted to O_i by O_{i-1}, and other information is inherent to O_i). Because of the encryption key, only the owner can transfer its value. And since p is a complete proof, due to the model in Section 3, the v_j's initial owner O_1 must have the legal proof of v_j, the induction is completed.*

Second, if p is a checkpoint proof, the proof process is similar to the above. But the first owner $O_{\bar{t}}$ of v_j in p is not the initial owner of v_j, thus the checkpoints is needed to prove the legality. According to the correctness of the CC algorithm (Theorem 1), as long as the transaction $txs_{u_{\bar{t}}}(O_{\bar{t}-1})$ in which $O_{\bar{t}}$ obtains v_j is not included in any checkpoint, it must be a legal transaction, i.e., v_j must belong to $O_{\bar{t}}$ at $B_{u_{\bar{t}}}$, then the induction still holds.

A.3 Proof of Theorem 3

Proof [Theorem 3] *First, the proof of Ownership is naturally included in the proof of Theorem 2, so we only need to prove Authenticity and Liquidity.*

For Authenticity, there is a fact that all initial owners of the values are legal (i.e., all values has one and only one initial owner). If there is a value v_i with two owners at the same time, then the previous owner of v_i must transfer v_i to two nodes. For such transaction, according to Theorem 2, an honest node can use **GetOwner** *to detect that it is illegal and reject this transaction. If a malicious owner transfers v_i to two other malicious nodes and successfully submits these two transactions to the ACC (this requires the leader to be malicious as well), this is possible, but it can keep for only one epoch at most. Because according to Theorem 1, both transactions will be marked illegal in the next checkpoint, and the owner of v_i will be marked as NA. Note that this temporary double-spending attack cannot be effective on any honest node since v_i cannot pass* **GetOwner** *(i.e., the double-spending value can only be transferred among the malicious nodes temporarily).*

For Liquidity, in Section 3, we assume that the message sent by the honest node has a bounded delay, and the AC algorithm satisfies the synchronous liveness, thus the delay from the transaction being submitted to its abstract being included by the ACC must be bounded by a constant.

The above conclusion is based on the success of the CC algorithm. Thus according to Theorem 1, EZchain fails with exponentially small probability.

APPENDIX B
OPTIMIZATION

Below we give some optimization solutions and analysis.

B.1 Choose the appropriate values for the transactions

In order for the node to allocate bandwidth resources more reasonably, the node should avoid the proof of a value (i.e., the length of the proof lp) being too long (or "old"), and preferentially choose the combined values (i.e., the values in the same txs) to pay. E.g., as shown in Figure 16: in case 1, for a transaction with 100 units, one pays 10 currencies of 10 units in the same inf and $lp = 1$; in case 2, for 10 transactions with 10 units, one pays 10 currencies of 10 units and $lp = 10$ separately. The total proof's size that case 2 needs to submit is 100 times that of case 1, and this waste will be amplified by m (the size of TC_l) times in the

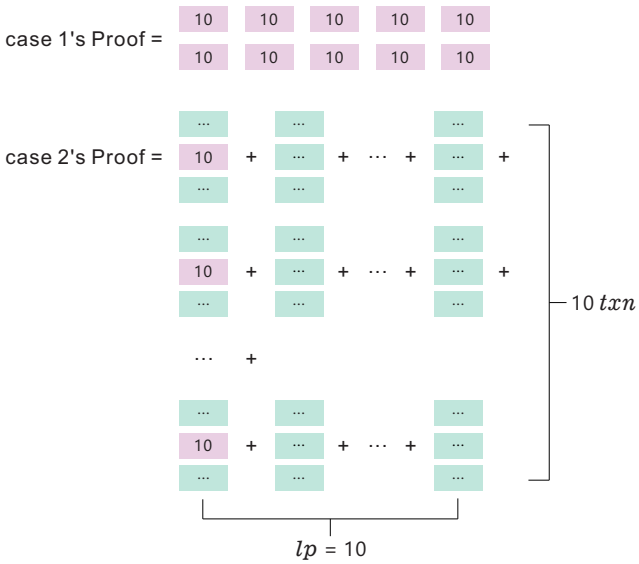

Fig. 16. This figure shows two payment cases (the purple block represents the value for payment and the green block is the part of the proof). Case 1 indicates that the values are aggregated in the inf, and they are transmitted together as one proof, which facilitates transmission to save bandwidth. Case 2 indicates that the values in the same inf are scattered in different proofs, transmissions occupy a large amount of bandwidth, and the longer the values' spans (i.e., the longer the lp), the more serious the waste.

CC algorithm. In order to reduce these wastes as much as possible, we give the following simple rules:

- The node records the proof's lp of each value held.
- The higher the lp, the higher the value's priority. In the same case, the priority of the combined values is higher.
- The higher the priority, the higher the probability that the value will be selected for the transaction.

Furthermore, in the analysis of CCPT, when the proof length of a value is longer, this means that the span of this value is longer, thereby causing h and f to increase (in Figure 4, the span becomes longer, then the probability that the transaction txn falls into this span becomes larger), i.e., the CCPT rises.

A feasible incentive scheme is: the transaction fee is adjusted according to the length of the proof, the long proof occupies more bandwidth resources, and also pays more transaction fees.

B.2 Choose the appropriate epoch's span T

The choice of the epoch's span T needs to consider multiple factors. In CCPT, the larger T is, the smaller f is (the larger the T, the smaller the probability that the value will be traded in the future epoch), but it also causes the k to increase (the larger the T, the more times the value is transferred in this epoch).

Below we model the optimal T simply, whose objective function is: $CCPT_T = m + h + k + m \cdot p$. And we denote the average transaction frequency of a node by σ (i.e., $T\sigma$ is the average number of transactions initiated by a node within time T); and the average frequency of the value's transfer by ρ (i.e., $T\rho$ is the average number of times a value was transferred during time T). Thus we have: $k = T\rho$, $f = h - T\sigma$ ($T \leq \frac{h}{\sigma}$), and $\rho = c \cdot \frac{T\sigma}{h}$, where c is a constant, since ρ should be proportional to σ and T and inversely proportional to h (according to the rules of Appendix B.1, the "older" the value, the easier it is to be traded). Then

$$CCPT_T = \frac{c\sigma}{h}T^2 - m\sigma T + mh + m + h.$$

In order to minimize $CCPT_T$, $T = \frac{mh}{2c}$.

B.3 A storage-saving scheme

In fact, the main chain (ACC) limits the upper bounds of some performance of EZchain. At this point, almost all of the sharding protocols are the same, since the calculation, consensus and storage of the main chain need to consume the corresponding resources. In EZchain, as transactions increase, then ACC grows, nodes need more storage space.

Below we present a compromise solution that solves this issue by sacrificing part of the bandwidth resources:

- The node x finds the set $S_x = \{B_i \mid \forall\ i\ s.t.\ \exists\ A_i(x)$ is included in $B_i\}$.
- For $\forall\ B_j \in S' = \{B_j \mid \forall\ j\ s.t.\ B_j \notin S_x\}$, x "compresses" B_j: $compress(B_j) = [\text{abstract of } B_j, list_j]$, where $list_j = \{k \mid \forall \text{ node } k\ s.t.\ \exists\ A_j(k)$ is included in $B_j\}$. After this operation, the abstract blocks which are not needed temporarily are all compressed by x.
- When the compressed B_j is needed, i.e., for the transaction $tx(v, y \rightarrow x)$, the proof of v is related to B_j. Then at this time x can ask y to provide B_j, since y must not compress B_j. Specifically, x can know which B_j needs to be transmitted according to $list_j$, and check the validity of B_j transmitted by y according to the abstract in $compress(B_j)$. In this way, x can get the full and valid B_j and perform the next verification, i.e., FC algorithm.

This solution can well reduce the storage requirements of EZchain. The uncompressed B_j that nodes need to store is much less than the full ACC. And this solution makes the storage requirements of the nodes almost independent of the number of transactions (although the compressed B_j still increases, yet it is almost negligible).

B.4 Save the bandwidth resources consumed by ACC

In the actual situations, since the consensus bandwidth resources required by the ACC are not negligible, then EZchain has a throughput upper bound (it is the same with almost all sharding protocols).

In theory, the abstract $A_i(x)$ in the abstract block B_i can contain any number of x's transactions. We call this number "abstract compression rate". Then the greater the abstract compression rate, the higher the system's throughput upper bound. Here we give an incentive strategy: EZchain's transaction fee does not charge for each transaction, but for each $A_i(x)$. Therefore the more transactions one $A_i(x)$ contains, the lower the transaction fee per transaction. This encourages the node x to package its own transactions into one $A_i(x)$ as much as possible.

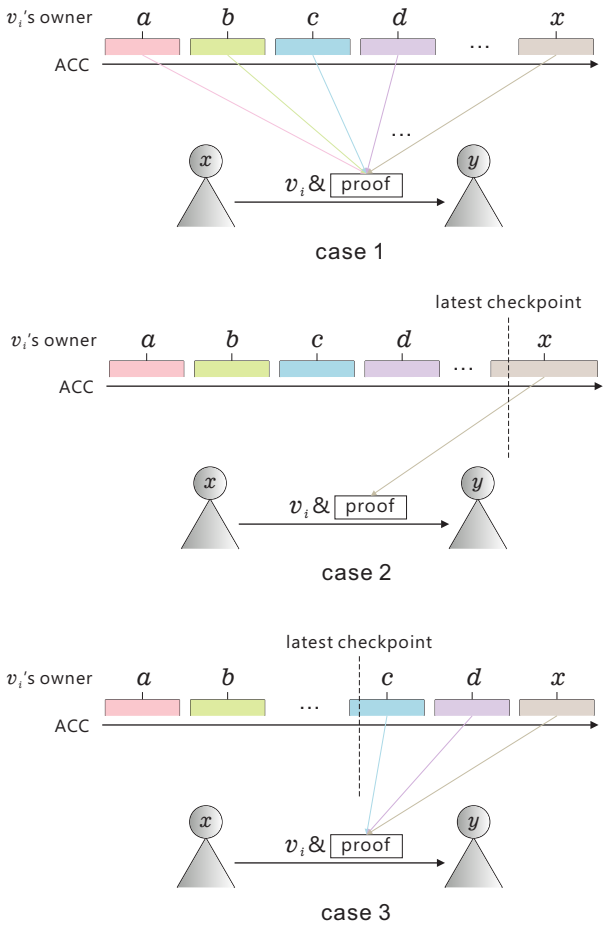

Fig. 17. Case 1 shows the $\mathcal{PV}$ that the node needs in VAPOR; Cases 2 and 3 show two different $\mathcal{PV}$ in our protocol respectively, the probability of case 2 is p and that of case 3 is $1-p$ (p is defined in Section 5.2.3). In case 2, $\mathcal{PV} \cap \mathcal{PB} = \mathcal{PV}$, and in case 3, $\mathcal{PV} \cap \mathcal{PB} \subset \mathcal{PV}$.

Appendix C
Analysis of different protocols' $E(M(*))$

First, we assume that all protocols have the same EM_{PB} (in fact, EZchain's EM_{PB} is smaller). The EM_B of our protocol and VAPOR's EM_B are the same. Our protocol's EM_{CB} is almost negligible compared to other items, because EM_{CB} only needs to list the illegal transactions' indices of each epoch. Thus, below we mainly analyze the $E(M(\mathcal{PV}))$ in our protocol and VAPOR. As shown in Figure 17, the nodes in VAPOR need to store all records since the birth of the value as the proof, since only in this way can the value's validity be guaranteed. Yet in our protocol, the nodes only need to store the record after the latest checkpoint, the value's validity is guaranteed by the checkpoint and the proof provided by the node.

In OmniLedger, nodes need to reserve enough storage to store transactions generated by the entire shard in one epoch and the state block. Algorand uses block certificates and sharding blocks, the former requires an additional storage of about 300 KBytes (this would need an additional 30% storage for 1 MByte blocks), the latter makes the nodes only store the corresponding shard's transactions.

Appendix D
The probability of the CC algorithm's failure

Theoretically, it is possible that more than $1/2$ nodes of the TC_l fail in the CC algorithm, and the probability is:

$$P = \sum_{i=\lceil \frac{k}{2} \rceil}^{k} C_k^i p^i (1-p)^{k-i},$$

where p is the AC algorithm's tolerance bound since the leader election of the AC algorithm is fair (Section 4.2), and k is the size of TC_l. According to the current one-minute block interval of most AC algorithms [10], [13], we assume $k = 1000$ for $T = 1440$ minutes, i.e., 1 day. Then we have:

- When $p = 1/3$ (e.g., Byzcoin), $P = 1.3306 \times 10^{-27}$.
- When $p = 1/4$ (e.g., Bitcoin), $P = 1.2814 \times 10^{-64}$.

Therefore, under reasonable parameter settings, the probability P of CC algorithm's failure is lower than 10^{-26}. Further, using the Stirling's approximation (i.e., $\lim_{n\to\infty} \frac{n!}{\sqrt{2\pi n}(n/e)^n}$), it can be proved that P becomes exponentially small and approaches zero as N increases.

Appendix E
DoS attacks on VAPOR and EZchain

VAPOR is mainly composed of two parts: the abstract chain consensus and the value verification. The former does not undertake any verification operation, i.e., the leader never verifies the transactions' content, and only needs to reach a consensus on the abstract chain in the whole network. This can result in malicious nodes sending invalid transactions' abstracts to consume bandwidth resources continuously (i.e., DoS attacks), and since such invalid transactions do not need to actually occur, counterfeiting does not require cost. This will make the transaction pool full of invalid abstracts, real abstracts cannot be confirmed in time, and a lot of bandwidth resources are wasted.

Yet, EZchain is not affected by the DoS attacks, because the node must submit the complete transaction when submitting its abstract (see Section 4.2). As long as the leader is not malicious, the illegal transaction will be ignored.

Appendix F
Analysis of the VAPOR's scalability

Although VAPOR claims that it is scale-out and its CCPT $= O(b) = o(N)$ [22]. Yet we think this does not exactly match the definition of scale-out, consider this problem: In VAPOR, a transaction txn is transmitted 50 times in a network of 100 nodes (i.e., txn only needs to be known by 50 nodes), but when the network scales up to 200 nodes, does txn still only need to be transmitted 50 times? If the spread of txn is independent of the network size, then VAPOR is indeed scale-out, since adding more nodes means more bandwidth resources for the connection, and the bandwidth consumed by each transaction is unchanged, the TPS will rise linearly as the network size increases.

But in fact, it is difficult to guarantee the assumption that "the spread of the transaction is not related to the

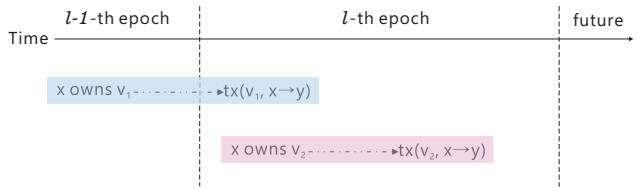

Fig. 18. The blue transaction means that v_1 is transferred for the first time in the l-th epoch; the red transaction means that x gets and transfers v_2 in the l-th epoch, so this transaction must not be the first transfer of v_2 in the l-th epoch.

network size in VAPOR". Consider the simplest case: The transfer of values is in accordance with the law of random walks. For the 100 new nodes, as the elapse of time, the values have a high probability of being transferred to them, i.e., the spread becomes wider. Then this assumption does not hold, thus VAPOR is not strictly scale-out, and its performance will decrease as the expansion of values' spreads.

In contrast, the active sharding protocol, such as OmniLedger, satisfies that whenever the system adds a shard, this part can provide the corresponding output, which satisfies scale-out. Although EZchain is based on VCB like VAPOR, as described in Section 5.2.1's Corollary 1, EZchain is scale-out as well.

Appendix G
Solution for the repeated transmissions in the CC algorithm

In order to solve the problem of repeated transmissions in the CC algorithm, we give the following scheme:

- For any legal transaction txn that occur in the l-th epoch, each $L_i (L_i \in TC_l)$ broadcasts the transaction only once. If txn need to be transmitted later (as part of prf) in TC_l, then only the txn's index is broadcast.
- For any value v_j, if the transaction $txn = tx(v_j, x \rightarrow y)$ is v_j being transferred for the first time in the l-th epoch, then txn with its proof need to be broadcast in TC_l (if an offline or malicious node does not broadcast this information in a fixed time, then the first honest node behind should broadcast txn with its proof); if txn is not the first time that v_j is transferred in the l-th epoch, then only txn needs to be broadcast in TC_l, and the proof of txn only needs to be broadcast the indices.

An intuitive example is presented in Figure 18. Below we give a brief description of the validity of this solution: If v_j is transferred for the first time in epoch l, then it is obviously feasible, we only consider the other case.

If v_j has been transferred in the epoch l, and honest node O_i^l transfers v_j to O_{i+1}^l at B_k. All transfer records of v_j in the epoch l are: O_1^l (gets v_j at $B_1^{l,j}$) $\rightarrow O_2^l$ (gets v_j at $B_2^{l,j}$) $\rightarrow \cdots \rightarrow O_i^l$ (gets v_j at $B_i^{l,j}$). Then there must be an honest node O_t^l to provide a complete proof $p =$**Proof**$(v_j, B_t^{l,j})$, otherwise O_i^l broadcast **Proof**$(v_j, B_i^{l,j})$. Without loss of generality, we set $t < i$, then O_i^l only needs to broadcast its own $txs_k(O_i^l)$, and $p = \{txs_k(O_i^l)\} \cup \{$the legal information provided by $O_h^l \mid t \leq h < i\}$ can be the proof that satisfies **GetOwner**$(v_j, B_k, p) = O_{i+1}^l$.

Appendix H
Passive sharding and VCB

Although VCB has the idea of passive sharding (PS), PS is a more generalized concept. Unlike active sharding, PS does not require an algorithm to perform the sharding process. The idea of PS is similar to "entities should not be multiplied unnecessarily", i.e., the message is only transmitted to a node when it needs this message, and before that, the node can be completely unaware of the existence of the message. In Type II protocols and even most sharding protocols, many messages are transmitted to the nodes that do not need them (these messages do not have any effect on the node's transactions, and the nodes only verify the legitimacy of these messages).

The sharding of the PS is reflected in the division of nodes by information. In VCB (or VAPOR), for a transaction, all nodes that need to know it are divided into the same subset, so that the network is divided into many subsets, and these subsets can intersect, which is different from the active sharding. How to divide these subsets is the core problem of the PS protocol. The division of EZchain and VAPOR is not optimal, i.e., there may still be redundant information between each subset (e.g., EZchain's solution for the redundant information of VCB: for transaction $txn_1 = tx(v, A \rightarrow B)$ and $txn_2 = tx(v, B \rightarrow C)$, then "$txn_2$ is legal" actually contains "txn_1 is legal", thus EZchain uses the checkpoint mechanism to reduce the redundant information between each subset of the original VCB).

In addition, the PS distributes the verification of the message to the message-related node itself, which effectively reduces the work and centralization of the validator (yet EZchain adds redundant verification operations, i.e., CC algorithm, for better performance).